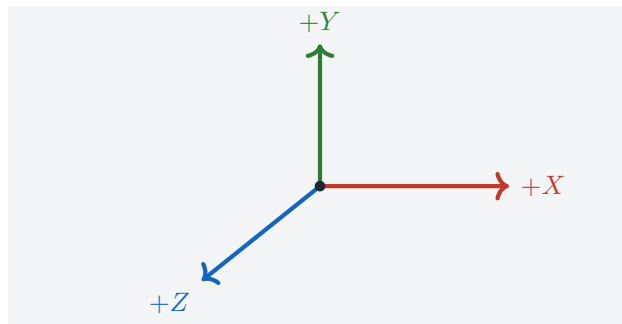


raylib

from Zero

A hands-on manual for someone who has
never drawn a single triangle in 3D



Window and frame loop · drawing · input · cameras
2D and 3D · collisions · and a silly little Doom at the end

Contents

I	Making a game	1
1	Before we start	2
1.1	What raylib actually is	2
1.2	What you need to know already	2
1.3	The files that come with this manual	2
1.4	What a build actually needs	3
1.5	How to read this manual	3
2	The shape of every raylib program	5
2.1	Immediate mode, and why nothing stays on screen	5
2.2	The skeleton	5
2.2.1	1. InitWindow	6
2.2.2	2. WindowShouldClose	6
2.2.3	3 and 4. BeginDrawing and EndDrawing	7
2.2.4	ClearBackground	7
2.3	Update and draw: keep them apart	7
2.4	Frame rate, and the two ways to control it	7
3	The screen: coordinates, colour, shapes	9
3.1	Where is (0,0)?	9
3.2	Corners and centres	9
3.3	Rectangle, the struct you will live in	10
3.4	Colour	10
3.5	The order of your draw calls is the layer order	10
3.6	Text	11
3.7	The shapes you actually use	11
4	Input	13
4.1	Three questions you can ask about a key	13
4.2	The mouse	13
4.3	Text input	14

5	Time, and why everything is multiplied by delta time	15
5.1	The problem	15
5.2	The fix	15
5.3	Reading a movement line	15
5.4	Timers	16
5.5	The one case where delta time bites back	16
6	Collisions in 2D: the rehearsal	17
6.1	Detecting	17
6.2	Resolving, which is the hard half	17
6.3	Keeping things on screen	18
7	Images, textures and sprites	19
7.1	Two things that sound identical and are not	19
7.2	The golden rule of loading	19
7.3	Drawing a texture, four ways	20
7.4	Sprite sheets	20
7.5	Where files are looked for	20
8	Camera2D: world coordinates and the Begin/End idea	22
8.1	The problem with drawing in pixels	22
8.2	Camera2D	22
8.3	The Begin/End pair, which is the real lesson	22
8.4	Going between the two worlds	23
8.5	Making the camera feel good	23
9	Into 3D: the three axes	24
9.1	One new number	24
9.2	Which way each axis points	24
9.3	The floor is the XZ plane	25
9.4	Objects sit on the floor, they do not live at zero	25
9.5	Units	25
10	The 3D camera	26
10.1	It is five numbers, and none of them are magic	26
10.2	fovy, or choosing a lens	27
10.3	Perspective versus orthographic	27
10.4	Begin and End, again	27
10.5	The ready-made cameras	28

11 Drawing 3D objects	29
11.1 The shapes that come in the box	29
11.2 Why everything looks flat	29
11.3 A fake shadow	30
11.4 Making something face the player	30
11.5 Models, when you outgrow cubes	30
11.6 The maths helpers	30
12 Your own first person camera	32
12.1 The two angles	32
12.2 From angles to a direction	32
12.3 From a direction to sideways	33
12.4 Moving	33
12.5 The four details that make or break the feel	33
12.6 Reading the mouse	33
12.7 The finishing touch: head bob	34
13 Collisions in 3D	35
13.1 The same idea, one more axis	35
13.2 Centre and size, not corners	35
13.3 Resolving, per axis, exactly as in 2D	36
13.4 Debugging: draw the boxes	36
13.5 Grids: the other way, and the fast one	36
13.6 Shooting: rays	37
14 Building a silly little Doom	38
14.1 The level is a piece of text	38
14.2 From map cell to world position	38
14.3 Reading the map once, at startup	39
14.4 The order of the frame	39
14.5 Walls	40
14.6 Shooting	40
14.7 Enemies	41
14.8 The HUD, or where the 2D course pays off	41
14.9 Things to change first	41

II	The rest of the library	43
15	Sound and music	44
15.1	One line before anything	44
15.2	The four things raudio gives you	44
15.3	Sounds	44
15.3.1	The trick that makes repeated sounds bearable	45
15.4	Music	45
15.5	Making sounds with no sound files	45
15.5.1	Wave and Sound are two separate allocations	46
15.6	Audio streams, for sound you generate live	46
15.7	Volume, and being polite	47
16	Shaders, and why your cubes look flat	48
16.1	What a shader actually is	48
16.2	The three steps, which never change	48
16.3	Uniforms: the only way to talk to the GPU	49
16.4	A fragment shader, line by line	49
16.5	Errors show up at run time, not compile time	50
16.6	Lighting, honestly	50
17	Meshes, models and materials	51
17.1	The three layers	51
17.2	Generating geometry	51
17.3	Putting a texture on something	52
17.4	Drawing	52
17.5	Loading real models	52
17.6	Animation	52
17.7	Billboards: the old trick, still useful	53
17.8	When to stop using DrawCube	53
18	Images, render textures and colour	54
18.1	Image work happens on the CPU	54
18.2	Generating images from nothing	54
18.3	Render textures: drawing into a texture	55
18.4	Texture filters and mipmaps	55
18.5	Colour helpers	56

19	Fonts and text	57
19.1	Loading a font properly	57
19.2	Drawing with it	57
19.3	Measuring, which is how you lay anything out	58
19.4	The string helpers	58
19.5	Unicode, briefly	58
19.6	Where to get fonts	59
20	The rest of rcore: saving, the window, other input	60
20.1	Saving and loading	60
20.2	Paths, and the one that actually matters	61
20.3	Dropped files	61
20.4	Random numbers	61
20.5	Logging	62
20.6	Managing the window at run time	62
20.7	Gamepads	62
20.8	Touch and gestures	63
20.9	Clipboard and compression	63
21	raymath	64
21.1	The three ideas worth understanding	64
21.1.1	Normalize: direction without length	64
21.1.2	Dot product: how much two directions agree	64
21.1.3	Cross product: perpendicular to both	65
21.2	The families	65
21.2.1	Utils	65
21.2.2	Vector2, Vector3, Vector4	65
21.2.3	Matrix	66
21.2.4	Quaternion	66
21.3	You do not have to use it	66
22	Where to go from here	68
22.1	What you have now	68
22.2	The examples are the next teacher	68
22.3	Three things this manual did not cover	68
22.4	The tools on raylib.com	69
22.5	Habits worth keeping	69
22.6	A last word	69

A	Cheat sheet	70
A.1	The skeleton	70
A.2	Window and timing	70
A.3	Drawing state	71
A.4	2D drawing	71
A.5	3D drawing	71
A.6	Input	71
A.7	Cameras	72
A.8	Collisions	72
A.9	raymath, the useful half	72
A.10	Audio	73
A.11	Shaders	73
A.12	Meshes, models and materials	73
A.13	Images and render textures	74
A.14	Files, saving and random	74
A.15	Loading and unloading	74
B	When something goes wrong	75
B.1	It will not compile or link	75
B.2	The window is there but the screen is black	75
B.3	The thing I drew is not there	75
B.4	It moves wrong	76
B.5	The collisions are wrong	76
B.6	It flickers	76
B.7	It runs at four frames per second	76
B.8	When none of that helps	77
C	The lessons, at a glance	78
C.1	course2d — the fundamentals	78
C.2	course3d — the 3D half	78
C.3	cheatsheet — one example per module (Part II)	79
C.4	Which chapter goes with which example	79
C.5	Controls, by lesson	80

Part I

Making a game

Before we start

1.1 What raylib actually is

raylib is a C library that hands you a window, a keyboard, a mouse, a sound card and a way to put coloured pixels on screen. That is it. It is not an engine. There is no editor, no scene graph, no entity system, no asset pipeline, no project format. You write a `main()`, you call some functions, you get a game.

That is the reason it feels strange at first if you are used to tools that hand you a scene and ask you to fill it in. In raylib *you* own the loop. Nothing happens that you did not write a line for.

The whole library is about 500 functions, and they are named so plainly that you can usually guess them: `DrawCircle`, `IsKeyDown`, `LoadTexture`, `CheckCollisionRecs`. There is no official reference manual, on purpose — the header file `raylib.h` *is* the documentation, one line of comment per function, and the examples are the tutorial.

THE KEY IDEA

If you remember one thing from this manual, remember this: the `raylib.h` header is the documentation. Open it. Search it. Every function you will ever use is in there with a one-line comment explaining it.

1.2 What you need to know already

C. Not advanced C — variables, `if`, `while`, functions, arrays and structs are enough. Every piece of code in this manual stays inside that. You do not need pointers-to-pointers, you do not need `malloc`, and you certainly do not need C++.

You do *not* need any 3D background. Chapters 9 to 14 assume you have never seen a 3D coordinate system in your life, and build it up from a single arrow on a page.

1.3 The files that come with this manual

This manual is not a thing to read in an armchair. Every chapter has a matching program you can run, change and break. They live in two folders next to it:

Folder	Part	What is in it
course2d/	I	the fundamentals: window, loop, shapes, input, collisions, camera
course3d/	I	the 3D half: axes, cameras, movement, collisions, the mini Doom
cheatsheet/	II	one example per module of the official raylib cheatsheet

Each folder has a `build.sh` that compiles and runs one of them:

```
./course2d/build.sh 1      # builds and runs course2d/lesson01_window.c
./course3d/build.sh 7      # builds and runs the mini Doom
./cheatsheet/build.sh 9    # builds and runs the audio module example
```

Part I is a path: read it in order and you end up with a game. Part II is a reference: each chapter stands alone, so read the one you need when you need it.

The script needs the raylib library to have been built once. If you have never run the project, do that first from the root folder:

```
make                      # builds bin/Debug/libraylib.a and the sample app
```

1.4 What a build actually needs

When people get stuck on day one, it is almost never the code — it is the compiler command. There is no magic in it, so here it is in full, the one that `build.sh` runs for you:

```
cc mygame.c                # your source file
-o mygame                  # the program to produce
-I build/external/raylib-master/src # where raylib.h lives
bin/Debug/libraylib.a      # the library itself
-lpthread -lm -ldl -lrt -lX11 # what raylib needs on Linux
```

Three ingredients, always: your `.c` file, the *header* so the compiler knows the function signatures, and the *library* so the linker can find the actual code. If you get `raylib.h: No such file` you got the `-I` wrong. If you get `undefined reference to InitWindow` you got the library path wrong. Those two errors mean two very different things and mixing them up costs beginners hours.

WATCH OUT

The order matters on Linux: the library goes *after* your `.c` file on the command line, never before. GNU `ld` resolves symbols left to right, and a library listed first has nothing left to resolve.

1.5 How to read this manual

Read a chapter, then run its lesson, then go back and change three numbers in the lesson until something breaks. That third step is the one that actually teaches you. Nothing here can be damaged: the worst that happens is a black window, and you press Escape.

The boxes mean:

WATCH OUT

A mistake almost everyone makes here. Reading these will save you an afternoon each.

THE KEY IDEA

The one sentence from the section that is worth carrying with you.

RUN IT

The lesson that shows this chapter running.

```
./course2d/build.sh 1
```

The shape of every raylib program

2.1 Immediate mode, and why nothing stays on screen

Here is the idea that everything else in raylib hangs from. Most graphical toolkits you have met are *retained mode*: you create a button object, you add it to a window, and it stays there. The toolkit remembers it and redraws it for you.

raylib is *immediate mode*. Nothing is remembered. There is no circle object. `DrawCircle()` does not create a circle — it paints circle shaped pixels, right now, and then forgets all about it. If you want the circle to still be there in a sixtieth of a second, you call `DrawCircle()` again.

THE KEY IDEA

Your program repaints the entire picture, from a blank canvas, sixty times a second. Everything you can see on screen is something you drew this frame. Everything you stop drawing disappears instantly.

This sounds wasteful and it is not: GPUs are extremely good at exactly this. And it buys you something precious for a beginner — there is no hidden state. If something is on screen, a line in your code put it there this frame. If something is missing, a line in your code did not run. Debugging graphics becomes reading your own draw calls in order.

2.2 The skeleton

Every raylib program, from the ten line example to the full game, has this shape:

```
#include "raylib.h"

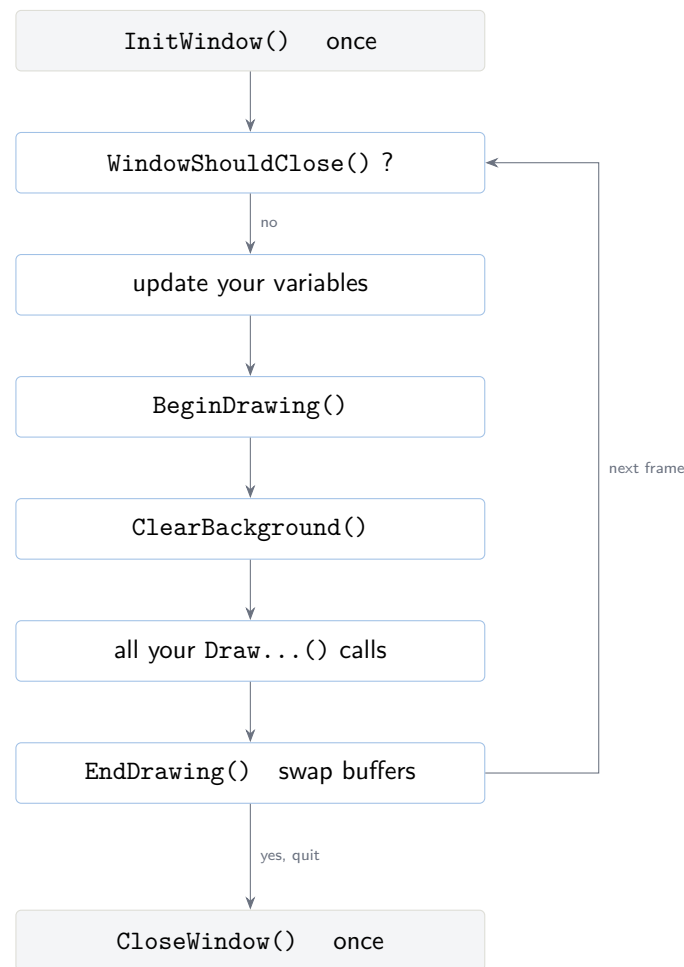
int main(void)
{
    InitWindow(800, 450, "My game"); // 1. once: open the window
    SetTargetFPS(60);

    while (!WindowShouldClose()) // 2. the frame loop
    {
        // UPDATE: change variables, draw nothing

        BeginDrawing(); // 3. start painting
        ClearBackground(RAYWHITE);
        DrawText("Hello", 20, 20, 20, DARKGRAY);
        EndDrawing(); // 4. show it
    }

    CloseWindow(); // 5. once: clean up
    return 0;
}
```

That is lesson 1 of the 2D course, near enough. Let us take the five parts apart.



2.2.1 1. InitWindow

`InitWindow(width, height, title)` creates the window and, more importantly, the OpenGL context behind it. Until this line runs, no other raylib function works — not loading a texture, not measuring text, nothing. Calling `LoadTexture()` before `InitWindow()` is a classic first-day crash.

Flags are the exception: `SetConfigFlags()` must be called *before* `InitWindow()`, because it configures how the window gets created. The two you will want:

```
SetConfigFlags(FLAG_VSYNC_HINT | FLAG_MSAA_4X_HINT);
InitWindow(800, 450, "My game");
```

`FLAG_VSYNC_HINT` syncs your drawing to the monitor's refresh so the image does not tear. `FLAG_MSAA_4X_HINT` turns on antialiasing so diagonal edges stop looking like staircases.

2.2.2 2. WindowShouldClose

`while (!WindowShouldClose())` is your game loop. The function returns true when the user presses Escape or clicks the window's close button, so the loop reads as "keep going until they want out".

Each pass through this loop is one *frame*. At 60 FPS the body of that loop runs 60 times a second, which gives you about 16 milliseconds to do everything.

2.2.3 3 and 4. BeginDrawing and EndDrawing

This pair is where the confusion usually is, so let us be precise.

`BeginDrawing()` says “I am about to paint”. It prepares the canvas for this frame. Every `Draw...`() call after it adds geometry to a batch that raylib is accumulating.

`EndDrawing()` does three jobs at once:

1. it flushes the batch — everything you queued actually gets sent to the GPU and drawn;
2. it swaps the buffers. You have been painting on a hidden canvas the whole time; this is the moment it becomes the visible one. That is *double buffering*, and it is why you never see a half-drawn frame;
3. it polls input events and waits, if needed, to hit your target frame rate.

WATCH OUT

Nothing appears on screen until `EndDrawing()` runs. If you put a `sleep()` or a long loop between `BeginDrawing()` and `EndDrawing()` hoping to see an animation, you will see nothing at all but a frozen window. Animation happens *across* frames, never inside one.

2.2.4 ClearBackground

The window still holds last frame’s pixels. `ClearBackground(RAYWHITE)` wipes them. Skip it and you get smears — which, incidentally, is a fun thing to try once on purpose: comment it out in lesson 1 and move something around.

2.3 Update and draw: keep them apart

Notice how the skeleton has a comment saying “UPDATE” before `BeginDrawing()`. That separation is not enforced by raylib, it is a discipline, and it is worth adopting from day one:

- **Update** — read input, move things, run the AI, check collisions. Change variables. Draw nothing.
- **Draw** — paint the current state of those variables. Change nothing.

When the two get tangled you end up with bugs where an object reacts to a collision that was resolved half a frame ago, and they are miserable to find. Keeping them apart costs nothing and every lesson in this course does it.

2.4 Frame rate, and the two ways to control it

`SetTargetFPS(60)` tells raylib to throttle the loop to at most 60 iterations per second. Without it your loop runs as fast as the machine allows, burning 100% of a CPU core to draw frames nobody will see.

`FLAG_VSYNC_HINT` does a similar job at a different level: it makes the buffer swap wait for the monitor. In practice, set both; they cooperate.

Two useful readings, both free:

```
int    fps = GetFPS();           // frames per second, as an integer
float  dt  = GetFrameTime();      // seconds the LAST frame took: ~0.0166 at 60fps
double t   = GetTime();          // seconds since InitWindow()
```

`GetFrameTime()` is the important one and it gets its own chapter (chapter 5). It is the difference between a game that behaves the same on every computer and one that does not.

RUN IT

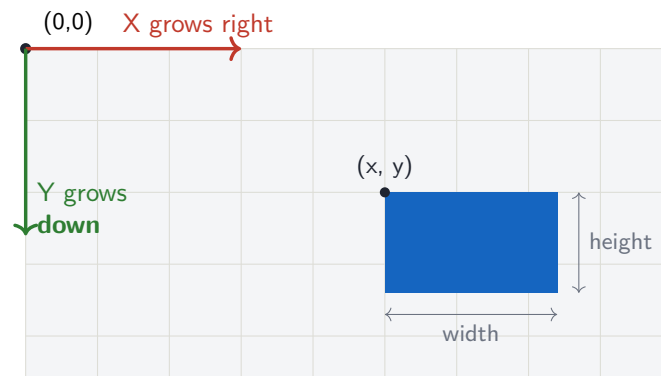
Lesson 1 prints the frame counter, the elapsed time and the frame duration, live. Watch the frame counter climb and realise that every one of those numbers was repainted from scratch.

```
./course2d/build.sh 1
```

The screen: coordinates, colour, shapes

3.1 Where is (0,0)?

Top left corner. X grows to the right, Y grows *downwards*.



`DrawRectangle(x, y, width, height, color)`

If you did maths at school, Y pointing down feels wrong. It comes from how screens were scanned, line by line from the top, and every 2D graphics API on earth inherited it.

WATCH OUT

In 3D (chapter 9) Y goes back to pointing *up*. Same letter, opposite direction, in the same program, sometimes on the same screen. When something moves the wrong way vertically, this is the first thing to check: which of the two worlds am I in?

Two helpers give you the current window size, and you should prefer them over hardcoding the numbers you passed to `InitWindow`, because the window can be resized:

```
int w = GetScreenWidth();
int h = GetScreenHeight();
DrawText("centred-ish", w/2 - 60, h/2, 20, DARKGRAY);
```

3.2 Corners and centres

`DrawRectangle` takes the **top-left corner**. `DrawCircle` takes the **centre**. That inconsistency has burned every raylib beginner, so hold on to it:

```
DrawRectangle(100, 100, 80, 40, MAROON); // corner at (100,100), 80x40
DrawCircle(100, 100, 30, DARKBLUE);     // CENTRE at (100,100), radius 30
```

If you want a rectangle centred at a point, you subtract half its size yourself:

```
DrawRectangle(cx - w/2, cy - h/2, w, h, MAROON);
```

You will write that line a thousand times. In the 3D chapters it comes back as `BoxFromCenter()`, which is the same idea with one more axis.

3.3 Rectangle, the struct you will live in

```
typedef struct Rectangle {
    float x;           // LEFT edge
    float y;           // TOP edge
    float width;
    float height;
} Rectangle;
```

Many functions take one of these instead of four loose numbers, and the collision functions all do. Get used to building them inline:

```
Rectangle player = { 100, 100, 40, 40 };
DrawRectangleRec(player, DARKBLUE);
DrawRectangleLinesEx(player, 2, BLACK);
```

3.4 Colour

A `Color` is four bytes, each 0 to 255:

```
typedef struct Color { unsigned char r, g, b, a; } Color;

Color myOrange = { 230, 120, 20, 255 }; // opaque
Color ghost    = { 230, 120, 20, 90 };  // mostly transparent
```

`a` is alpha: 255 is solid, 0 is invisible. raylib ships about twenty-five named colours — `RAYWHITE`, `DARKGRAY`, `MAROON`, `SKYBLUE`, `GOLD`, `BLANK` (fully transparent) and friends — and one very handy helper:

```
DrawRectangle(10, 10, 100, 100, Fade(BLUE, 0.5f)); // BLUE at 50% alpha
```

`Fade` returns a copy of the colour with the alpha scaled. Use it for shadows, overlays, ghosts and dimmed HUDs instead of writing the struct out.

WATCH OUT

When you write a colour inline you need the cast, because C needs to know what the braces mean: `0,20,25,255DrawRectangle(0,0,10,10,(Color)2)`. Forgetting the `(Color)` gives you a confusing compiler error about initialisation.

3.5 The order of your draw calls is the layer order

There is no z-index, no layer system and no sorting. Whatever you draw last is on top. Painting a picture, literally.

```
DrawRectangle(640, 340, 100, 100, DARKPURPLE); // painted first: underneath
DrawCircle(700, 400, 45, GOLD);                // painted last: on top
```

That means the structure of your draw section is your scene's depth. Background first, then the world, then characters, then effects, then the HUD last of all. When something is mysteriously invisible, nine times out of ten it is being drawn *before* something opaque.

3.6 Text

```
DrawText("Hello", 20, 20, 20, DARKGRAY); // text, x, y, fontSize, color
int width = MeasureText("Hello", 20);     // how wide would it be?
```

MeasureText is how you centre text, since there is no alignment parameter:

```
const char *txt = "GAME OVER";
DrawText(txt, GetScreenWidth()/2 - MeasureText(txt, 60)/2, 200, 60, RED);
```

And to print variables, TextFormat is raylib's sprintf that manages the buffer for you:

```
DrawText(TextFormat("score: %i   time: %.2f", score, GetTime()), 20, 20, 20, BLACK);
```

WATCH OUT

TextFormat writes into an internal rotating buffer. It is perfect for drawing this frame and wrong for storing: never keep the returned pointer across frames, copy the string if you need it later.

3.7 The shapes you actually use

There are about forty drawing functions; these are the ones that cover almost everything:

Function	Notes
DrawRectangle(x,y,w,h,c)	corner based
DrawRectangleRec(rec,c)	takes a Rectangle
DrawRectangleLinesEx(rec,thick,c)	outline with thickness
DrawRectangleRounded(rec,round,seg,c)	rounded corners
DrawCircle(cx,cy,r,c)	centre based
DrawCircleLines(cx,cy,r,c)	outline
DrawLine(x1,y1,x2,y2,c)	one pixel thin
DrawLineEx(v1,v2,thick,c)	with thickness
DrawTriangle(v1,v2,v3,c)	counter-clockwise vertices
DrawPoly(centre,sides,r,rot,c)	regular polygon

The naming pattern repeats across the whole library and is worth learning as a rule rather than as forty separate facts:

- plain name, e.g. DrawCircle — the simple version, loose **int** coordinates;

- **V** suffix, e.g. `DrawCircleV` — same thing but taking a `Vector2`;
- **Rec** suffix — takes a `Rectangle`;
- **Ex** suffix — extended: extra parameters like thickness;
- **Pro** suffix — all the knobs: origin, rotation, scaling;
- **Lines** in the name — outline instead of filled.

Once you see that, you can guess most of raylib without looking anything up.

WATCH OUT

`DrawTriangle` wants its three vertices in counter-clockwise order. Give them clockwise and the triangle is facing away from you, so it gets culled and you see nothing at all. If a triangle is invisible, swap two of its vertices before you look for any other bug.

RUN IT

Lesson 2 draws the coordinate grid, every basic shape labelled, alpha blending, and lets you swap the draw order with SPACE to watch the layering change.

```
./course2d/build.sh 2
```

Input

4.1 Three questions you can ask about a key

This is short but it matters enormously, because picking the wrong one of the three is the most common beginner bug in any engine.

Function	True when	Use it for
<code>IsKeyDown(KEY_D)</code>	every frame it is held	walking, aiming, holding
<code>IsKeyPressed(KEY_D)</code>	the single frame it goes down	jump, shoot, menus
<code>IsKeyReleased(KEY_D)</code>	the single frame it comes up	charge-and-release

Hold a key for one second at 60 FPS: `IsKeyDown` is true sixty times, `IsKeyPressed` is true once.

WATCH OUT

Use `IsKeyDown` for a menu and your cursor jumps sixty entries in one second. Use `IsKeyPressed` for walking and your character teleports one step per tap and refuses to walk. When input “feels broken”, check which of the two you used before anything else.

Key names are the obvious ones: `KEY_A` to `KEY_Z`, `KEY_ZERO` to `KEY_NINE`, `KEY_SPACE`, `KEY_ENTER`, `KEY_LEFT_SHIFT`, `KEY_LEFT_CONTROL`, `KEY_UP`, `KEY_LEFT`, `KEY_TAB`, `KEY_ESCAPE`. They are all listed in `raylib.h` under `KeyboardKey`.

WATCH OUT

`KEY_ESCAPE` closes the window by default — that is what `WindowShouldClose()` watches. If you want Escape to open a pause menu instead, turn the default off with `SetExitKey(KEY_NULL)` and handle it yourself.

4.2 The mouse

```
Vector2 mouse = GetMousePosition();    // in SCREEN pixels
float   wheel = GetMouseWheelMove();    // -1, 0 or +1 sized steps
Vector2 delta = GetMouseDelta();        // how far it moved THIS frame

if (IsMouseButtonPressed(MOUSE_BUTTON_LEFT)) { /* click */ }
if (IsMouseButtonDown(MOUSE_BUTTON_RIGHT))  { /* held */ }
```

The same pressed/down/released trio applies to buttons.

`GetMouseDelta()` is the one that makes first person cameras possible. It reports movement rather than position, which is exactly what you want when the cursor is locked in place and can no longer move:

```
DisableCursor();    // hide the cursor and lock it to the window centre
EnableCursor();     // give it back to the user
```

With the cursor disabled, `GetMousePosition()` stops being useful (the pointer is pinned) but `GetMouseDelta()` keeps reporting how much the user moved the physical mouse. Chapter 12 turns that into a camera.

WATCH OUT

Always call `EnableCursor()` before `CloseWindow()`, or a crash can leave the user's pointer captured and invisible. Every lesson in the 3D course does this.

4.3 Text input

For a name entry field you do not want key codes, you want characters, with the keyboard layout and the shift key already applied:

```
int c = GetCharPressed();
while (c > 0)
{
    if ((c >= 32) && (c <= 125) && (length < MAX)) name[length++] = (char)c;
    c = GetCharPressed();    // the queue may hold several this frame
}
if (IsKeyPressed(KEY_BACKSPACE) && length > 0) name[--length] = '\0';
```

It is a queue, so you loop until it is empty. Handle backspace with `IsKeyPressed` because it is an action, not a character.

RUN IT

Lesson 3 shows the pressed-versus-down counters side by side. Hold SPACE for a second and compare the two numbers.

```
./course2d/build.sh 3
```

Time, and why everything is multiplied by delta time

5.1 The problem

Here is a piece of code that looks completely reasonable and is wrong:

```
if (IsKeyDown(KEY_D)) player.x += 5;      // WRONG
```

It says “move five pixels per frame”. On a 60 Hz monitor that is 300 pixels a second. On a 144 Hz monitor it is 720. On a machine struggling at 30 FPS it is 150. Your game is a different game on every computer, and if the frame rate stutters, the physics stutter with it.

5.2 The fix

`GetFrameTime()` returns how long the previous frame took, in seconds. People universally call it *delta time*, or `dt`. At 60 FPS it is about 0.0166; at 120 FPS about 0.0083.

```
float dt = GetFrameTime();
float speed = 300.0f;           // PIXELS PER SECOND

if (IsKeyDown(KEY_D)) player.x += speed*dt;  // RIGHT
```

At 60 FPS that adds $300 \times 0.0166 = 5$ pixels. At 120 FPS it adds $300 \times 0.0083 = 2.5$ pixels, twice as often. Same distance per second, on every machine.

THE KEY IDEA

Every speed you write down is per *second*, and every time you apply it you multiply by `dt`. Not sometimes — always. Movement, rotation, timers, fading, reloading, animation. If a number changes over time, it is multiplied by `dt`.

5.3 Reading a movement line

Once the habit sticks, your constants start documenting themselves:

```
float walkSpeed = 4.0f;        // metres per second -> a person walking
float runSpeed  = 8.0f;        // metres per second -> a person running
float turnSpeed = 90.0f;       // degrees per second -> a quarter turn each second
float reload    = 1.5f;        // seconds
```

Compare that with “0.066 per frame” and you can see which version you would rather debug at midnight.

5.4 Timers

The same logic drives anything that happens after a delay. Add `dt` to a counter and check it:

```
float shootCooldown = 0.0f;

// update
if (shootCooldown > 0.0f) shootCooldown -= dt;

if (IsMouseButtonPressed(MOUSE_BUTTON_LEFT) && shootCooldown <= 0.0f)
{
    Shoot();
    shootCooldown = 0.25f;    // four shots per second, on any machine
}
```

Animation is the same pattern, and lesson 5 uses exactly this shape to step through a sprite sheet.

WATCH OUT

Do not use `GetTime()` for this. `GetTime()` is absolute seconds since startup: fine for a clock, awkward for “0.25 seconds from now” because you have to remember when now was. Counting down a float with `dt` is simpler and pauses for free when you stop updating it.

5.5 The one case where delta time bites back

If the game freezes for a second — a big file loads, you dragged the window, you hit a breakpoint — the next `dt` is huge. Everything moves a second’s worth in one frame, and fast objects teleport straight through walls, because the collision check only looks at where they *landed*, not the path they took.

The standard defence is one line, and it is why you often see this in real games:

```
float dt = GetFrameTime();
if (dt > 0.05f) dt = 0.05f;    // never simulate more than 50 ms in one step
```

The game slows down briefly instead of exploding. Good enough for everything you will write for a long while.

RUN IT

Lesson 3 puts a green square that uses `dt` next to a red square that does not, and drives both from the same keys.

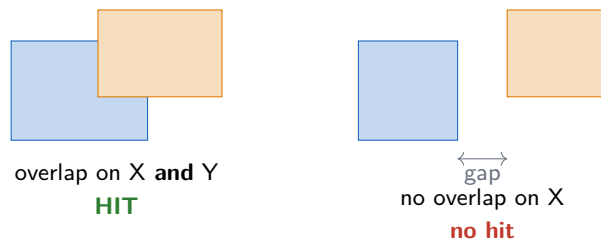
```
./course2d/build.sh 3
```

Collisions in 2D: the rehearsal

Everything in this chapter comes back, almost unchanged, in 3D. Learn it here where you can see it, and chapter 13 will feel like a rerun.

6.1 Detecting

Two rectangles overlap if they overlap on **both** axes at once. One axis where they do not overlap is enough to guarantee they are apart:



```
bool overlapX = (a.x < b.x + b.width) && (a.x + a.width > b.x);
bool overlapY = (a.y < b.y + b.height) && (a.y + a.height > b.y);
bool hit = overlapX && overlapY;
```

raylib gives you that in one call, and a few relatives:

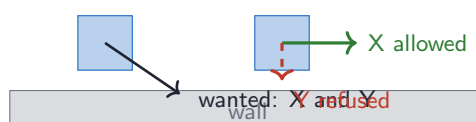
```
CheckCollisionRecs(recA, recB);           // box vs box
CheckCollisionCircles(centreA, rA, centreB, rB); // circle vs circle
CheckCollisionCircleRec(centre, radius, rec); // circle vs box
CheckCollisionPointRec(GetMousePosition(), button); // is the mouse over it?
```

That last one is how you write a button without any UI library: a `Rectangle`, a `CheckCollisionPointRec`, and `IsMouseButtonPressed`. Three lines, and you own it completely.

6.2 Resolving, which is the hard half

Knowing that you hit a wall does not stop you. The naive fix — “if I hit something, do not move at all” — produces a game that feels awful: walk into a wall at a slight angle and you stop dead instead of sliding along it.

The trick is to treat the axes separately:



```
// 1. try the X move on its own
Rectangle attempt = player;
attempt.x += dx;
if (!HitsAnyWall(attempt)) player.x = attempt.x;    // otherwise: keep old x

// 2. try the Y move on its own
attempt = player;
attempt.y += dy;
if (!HitsAnyWall(attempt)) player.y = attempt.y;    // otherwise: keep old y
```

Walking diagonally into a horizontal wall, the Y step is refused and the X step is allowed, so you slide along it. That is the whole secret, and it is the same code in 3D with *z* in place of *y*.

THE KEY IDEA

Detect with one call. Resolve one axis at a time. Two moves, two checks, never both at once.

6.3 Keeping things on screen

The cheapest collision of all, and one every game needs:

```
if (player.x < 0) player.x = 0;
if (player.y < 0) player.y = 0;
if (player.x + player.width > GetScreenWidth()) player.x = GetScreenWidth() -
    player.width;
if (player.y + player.height > GetScreenHeight()) player.y = GetScreenHeight() -
    player.height;
```

Note it clamps *after* the move rather than refusing it. For a wall you want to refuse the move; for a screen edge, clamping feels better because the player keeps sliding along the border.

RUN IT

Lesson 4 has a little maze. SPACE switches the walls between “solid” (resolved per axis) and “ghost” (detected only) so you can feel the difference.

```
./course2d/build.sh 4
```

Images, textures and sprites

7.1 Two things that sound identical and are not

	Image	Texture2D
Lives in	normal RAM (CPU)	video memory (GPU)
You can	read and edit pixels	draw it, fast
You cannot	draw it	edit single pixels
Typical use	load, resize, recolour	everything you see

The usual flow is: get an **Image**, upload it once, throw the **Image** away, draw the **Texture2D** forever.

```
Image img = LoadImage("player.png");      // into RAM
ImageResize(&img, 64, 64);                  // cheap to edit here
Texture2D tex = LoadTextureFromImage(img); // upload to the GPU
UnloadImage(img);                           // the RAM copy is dead weight now
```

If you do not need to touch the pixels, `LoadTexture("player.png")` does all four steps for you.

7.2 The golden rule of loading

WATCH OUT

Load **outside** the loop, unload **after** the loop. Calling `LoadTexture()` inside the frame loop uploads a fresh copy to the GPU sixty times a second. A minute later you are out of video memory and you have no idea why.

```
Texture2D tex = LoadTexture("player.png"); // once

while (!WindowShouldClose())
{
    BeginDrawing();
        DrawTexture(tex, 100, 100, WHITE); // every frame: just draws
    EndDrawing();
}

UnloadTexture(tex); // once
CloseWindow();
```

The same rule covers fonts, sounds, music and models. Every `Load...` has a matching `Unload...`, and they belong on either side of the loop.

7.3 Drawing a texture, four ways

```
DrawTexture(tex, x, y, WHITE);           // plain, at a position
DrawTextureV(tex, position, WHITE);      // same, Vector2
DrawTextureEx(tex, position, rotation, scale, WHITE); // rotated and scaled
DrawTextureRec(tex, source, position, WHITE); // just a PIECE of it
DrawTexturePro(tex, source, dest, origin, rot, WHITE); // everything at once
```

That `WHITE` at the end is the *tint*: the texture's colours get multiplied by it. `WHITE` means "unchanged". Pass `RED` and everything goes red; pass `Fade(WHITE, 0.5f)` and the sprite is half transparent. This is how you flash a character on damage without a second image.

7.4 Sprite sheets

An animation is not many files, it is one image holding many frames side by side, plus a `Rectangle` that says which frame to show:



```
Rectangle source = { frame*FRAME_SIZE, 0, FRAME_SIZE, FRAME_SIZE };
DrawTextureRec(sheet, source, position, WHITE);
```

And you advance frame on a timer, never on a frame counter, for the reason chapter 5 gave:

```
frameTimer += dt;
while (frameTimer >= frameDuration)
{
    frameTimer -= frameDuration;
    frame = (frame + 1) % FRAME_COUNT;
}
```

WATCH OUT

To flip a sprite horizontally, make the source rectangle's width *negative*. It reads like a bug and it is the intended API: `source.width *= -1` and your character faces the other way, with no second image and no rotation.

7.5 Where files are looked for

Paths are relative to the *working directory*, which is wherever you launched the program from, not where the executable sits. That is why running from an IDE sometimes finds your `.png` and running from a terminal does not.

This project ships a helper for exactly that, in `src/resource_dir.h`:

```
SearchAndSetResourceDir("resources"); // find it and chdir there
Texture2D tex = LoadTexture("wabbit_alpha.png");
```

WATCH OUT

A texture that failed to load is not a crash: you get a texture with `id == 0` and you draw nothing, silently. If a sprite is invisible, check `IsValid(tex)` before suspecting your draw code.

RUN IT

Lesson 5 builds a six frame sprite sheet in memory — no external files — and plays it back, showing the whole sheet, the current frame, and the source rectangle moving across it.

```
./course2d/build.sh 5
```

Camera2D: world coordinates and the Begin/End idea

8.1 The problem with drawing in pixels

Everything so far has been drawn at screen pixels. That works until the world is bigger than the window. Then you find yourself writing `DrawRectangle(enemy.x - scrollX, enemy.y - scrollY, ...)` on every single draw call, and forgetting the subtraction on one of them.

The fix is to stop thinking in pixels. Store everything in **world** coordinates — metres, tiles, whatever your game is made of — and let a camera decide which part of the world the window is looking at.

8.2 Camera2D

```
typedef struct Camera2D {
    Vector2 offset;      // where the target lands ON SCREEN (usually the middle)
    Vector2 target;      // the WORLD point the camera is centred on
    float rotation;      // degrees; rotates the whole world
    float zoom;          // 1.0 normal, 2.0 twice as big. Never 0.
} Camera2D;
```

The pair that confuses people is `target` and `offset`. Read it as a sentence: “*the world point target is displayed at the screen position offset*”. So to centre the camera on the player you set `target` to the player’s world position and `offset` to the middle of the window:

```
camera.target = player;                                // world
camera.offset = (Vector2){ GetScreenWidth()/2.0f, GetScreenHeight()/2.0f };
camera.zoom = 1.0f;
```

WATCH OUT

`zoom` defaults to 0 if you write `Camera2D camera = 0` and forget to set it — and a zoom of zero scales the entire world down to nothing. Black screen, no error, no clue. If your `Camera2D` shows nothing, this is why, every time.

8.3 The Begin/End pair, which is the real lesson

```
BeginMode2D(camera);
    // WORLD coordinates in here. The camera transforms everything.
    DrawRectangle(player.x, player.y, 30, 30, BLUE);
EndMode2D();
```

```
// SCREEN pixels again out here. Nothing below moves with the camera.  
DrawText("HEALTH 100", 20, 20, 20, RAYWHITE);
```

THE KEY IDEA

Inside `BeginMode2D/EndMode2D` you are in world space and the camera moves things around. Outside, you are in screen pixels and nothing moves. The HUD goes outside. Always.

This is worth dwelling on, because in three chapters' time you will meet `BeginMode3D / EndMode3D` and it is the same idea with a different camera: the `Begin` pushes a transform, the `End` pops it. Your health bar sits outside the pair in both cases, for exactly the same reason.

You can even nest them in one frame: draw the world in 3D, end the 3D mode, draw a 2D minimap and the HUD. That is precisely what the mini Doom in chapter 14 does.

8.4 Going between the two worlds

Sooner or later you need to know what the mouse is pointing at in world coordinates — to click an enemy, to place a tile:

```
Vector2 worldPoint = GetScreenToWorld2D(GetMousePosition(), camera);  
Vector2 screenPoint = GetWorldToScreen2D(enemy.position, camera);
```

The second one is how you put a floating name tag above a character: compute where the character is on screen, then draw text there *outside* the `Begin/End` pair so it keeps a constant size no matter the zoom. The 3D course uses the 3D version of this same function to label the axes in lesson 1.

8.5 Making the camera feel good

A camera nailed to the player is correct and slightly nauseating. Real games lag behind a little. One line, using `at` as always:

```
camera.target.x += (player.x - camera.target.x)*5.0f*dt;  
camera.target.y += (player.y - camera.target.y)*5.0f*dt;
```

Each frame the camera closes a fraction of the gap towards the player. Larger number, tighter follow. This is called exponential smoothing and it is worth knowing because it makes almost anything feel better: camera follow, aiming, health bars catching up to the real value.

RUN IT

Lesson 6 gives you a world far bigger than the window, with zoom and rotation, and a red crosshair drawn *outside* the `Begin/End` pair so you can watch it refuse to move with the world.

```
./course2d/build.sh 6
```

Into 3D: the three axes

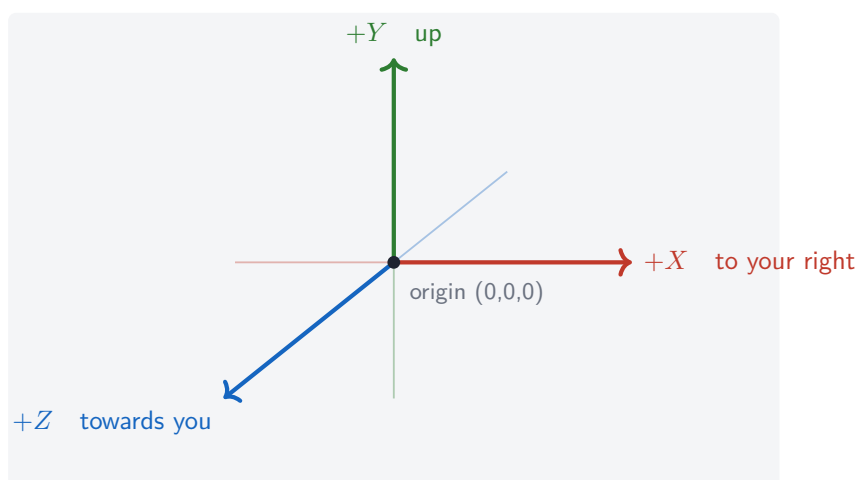
9.1 One new number

A 3D point is a 2D point with a third number bolted on. That is genuinely all that changes at the level of data:

```
typedef struct Vector2 { float x, y; } Vector2;
typedef struct Vector3 { float x, y, z; } Vector3;
```

What changes is what those numbers *mean*, and that is where the disorientation comes from, so let us be very clear about it.

9.2 Which way each axis points



Axis	Negative	Positive	Think of it as
X	left	right	sideways
Y	down	up	height
Z	far away	towards you	depth

This is a *right-handed* system, the same as OpenGL. If you want to check it physically: point your right thumb along +X and your index finger along +Y, and your middle finger points along +Z, out of the screen at you.

WATCH OUT

Y now points **up**. In the 2D half of this manual it pointed **down**. Both are true, in their own space, and both are live in the same program — a 3D game with a HUD uses both in the same frame. When something moves vertically the wrong way, ask yourself which space that line is in.

9.3 The floor is the XZ plane

This is the sentence that makes 3D click for most people.

Height is Y. So a floor — a flat surface at a constant height — is all the points where Y is the same, usually zero. The two axes left over, X and Z, are the ones you walk around on.

- A top-down map of your level only needs X and Z. That is why the minimap in chapter 14 maps world X to screen X and world Z to screen Y.
- `DrawGrid(20, 1.0f)` draws its grid on the XZ plane at $Y = 0$, because that is the floor.
- Walking means changing X and Z. Jumping means changing Y.

THE KEY IDEA

X and Z are the ground. Y is how high you are. A character walking around a room is a 2D problem in X and Z, with a Y that mostly stays at one value.

9.4 Objects sit on the floor, they do not live at zero

`DrawCube` takes the **centre** of the cube. So a cube of height 2 resting on the floor has its centre at $Y = 1$, not $Y = 0$. Put it at zero and half the cube is buried underground.

```
DrawCube((Vector3){ 0, 1, 0 }, 2, 2, 2, MAROON);    // a 2x2x2 cube ON the floor
```

And mind the parameter order, which quietly encodes the axes:

```
DrawCube(centre, width, height, length, colour);
//           ^X      ^Y      ^Z
```

width is the size along X, height along Y, length along Z. There is a `DrawCubeV` that takes a `Vector3` size instead, which removes the ambiguity, and it is the one worth preferring once the axes are second nature.

9.5 Units

raylib has no opinion about what one unit means. Pick one and keep it. This course uses **one unit = one metre**, so every number you write is something you can picture:

```
#define EYE_HEIGHT  1.7f    // a person's eyes
float walkSpeed = 4.0f;    // metres per second, a brisk walk
```

Once your constants are in metres and seconds, you can spot a wrong number just by reading it.

RUN IT

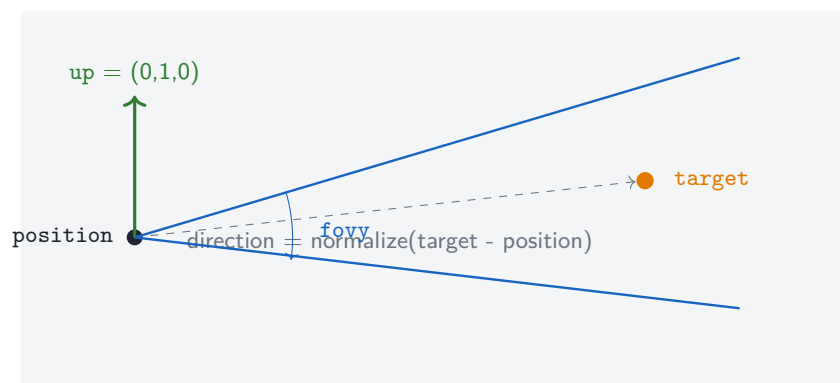
Lesson 1 of the 3D course draws the three axes in their standard colours — X red, Y green, Z blue — and lets you jump the camera onto each axis in turn with the keys 1 to 4.

```
./course3d/build.sh 1
```

The 3D camera

10.1 It is five numbers, and none of them are magic

```
typedef struct Camera3D {
    Vector3 position;    // where the eye is
    Vector3 target;      // the world point it is looking at
    Vector3 up;          // which way is "up" for the camera
    float fovy;          // vertical field of view, in DEGREES
    int projection;      // CAMERA_PERSPECTIVE or CAMERA_ORTHOGRAPHIC
} Camera3D;
```



Notice what is *not* in that struct: there is no field for the direction the camera is facing, and no rotation angles. The direction is worked out from the two points:

$$\text{direction} = \text{normalize}(\text{target} - \text{position})$$

Which gives you the two operations you will use forever:

- **To turn the camera:** move the `target`, leave `position` alone.
- **To move the camera:** move `position` *and* `target` by the same amount, so it does not turn while it travels.

WATCH OUT

Move only `position` and the camera swings round to keep staring at the old target, like a security camera tracking you. That is occasionally what you want and usually a bug.

10.2 fovy, or choosing a lens

`fovy` is the vertical angle the camera can see, in degrees. It behaves exactly like a camera lens:

fovy	Feels like
20–30	telephoto: flat, compressed, distant
45	neutral, good for inspecting an object
60–75	the normal first person game range
90+	fisheye: fast, dizzying, very wide

Doom-like shooters tend to sit around 70 to 90, which is why the mini Doom in chapter 14 uses 75.

10.3 Perspective versus orthographic

`CAMERA_PERSPECTIVE` is what your eyes do: distant things look smaller, parallel lines converge.

`CAMERA_ORTHOGRAPHIC` removes that. Distance stops affecting size and parallel lines stay parallel. This is the look of isometric strategy games, of CAD, and of 2D games faking depth.

WATCH OUT

When you switch to orthographic, `fovy` stops meaning degrees and starts meaning *how many world units fit vertically on screen*. Leave it at 45 and your world is 45 metres tall in the window — everything is a speck. Set something like 10 and it looks sane again.

10.4 Begin and End, again

```
BeginMode3D(camera);
    DrawGrid(20, 1.0f);
    DrawCube((Vector3){ 0, 1, 0 }, 2, 2, 2, MAROON);
EndMode3D();

// back to 2D screen pixels: the HUD
DrawText("HEALTH 100", 20, 20, 20, RAYWHITE);
DrawFPS(10, 10);
```

Exactly the same contract as `BeginMode2D` in chapter 8: inside is world space, outside is screen pixels. Your crosshair, health bar, ammo counter and minimap all go outside, and that is why they stay nailed to the window while the world moves.

There is one extra thing `BeginMode3D` does that the 2D version does not: it enables the **depth buffer**. In 2D the draw order decided what was on top; in 3D the GPU compares distances per pixel, so a cube behind another cube is hidden correctly no matter which order you drew them in. You get that for free.

WATCH OUT

Transparency is the exception. The depth buffer works per pixel, and a half transparent surface writes depth as if it were solid, which can hide things behind it. If you draw transparent objects, draw them after the opaque ones, furthest first. You will not hit this for a while, but when a glass wall makes things vanish, this is why.

10.5 The ready-made cameras

Before writing your own, try raylib's:

```
UpdateCamera(&camera, CAMERA_FREE);           // fly around: WASD, mouse, wheel
UpdateCamera(&camera, CAMERA_ORBITAL);        // spins around the target by itself
UpdateCamera(&camera, CAMERA_FIRST_PERSON);   // classic FPS
UpdateCamera(&camera, CAMERA_THIRD_PERSON);   // follows the target from behind
```

One line, in the update section, and the camera works. For first and third person, call `DisableCursor()` first so the mouse is captured.

They are perfect for looking at a model or prototyping a scene, and they run out of road the moment you want collisions, crouching, recoil, a vehicle, or a camera that does anything specific to your game. That is the subject of the next chapter.

RUN IT

Lesson 2 of the 3D course lets you move each of the five camera fields with the keyboard and watch the numbers; lesson 3 cycles the four built-in modes with TAB.

```
./course3d/build.sh 2    and    ./course3d/build.sh 3
```

Drawing 3D objects

11.1 The shapes that come in the box

raylib has no 3D modelling tools, but it has enough primitives to build a whole game out of, and every one of them is a single call:

```
DrawCube(centre, width, height, length, colour);    // sizes are X, Y, Z
DrawCubeV(centre, size, colour);                    // same, size as Vector3
DrawCubeWires(centre, w, h, l, colour);             // just the edges
DrawSphere(centre, radius, colour);
DrawCylinder(bottomCentre, radiusTop, radiusBottom, height, sides, colour);
DrawPlane(centre, size2D, colour);                  // a flat XZ quad
DrawLine3D(from, to, colour);
DrawGrid(slices, spacing);                          // the XZ floor grid
```

A few things worth knowing before you use them:

- `DrawCube` takes the **centre**, like `DrawCircle` did in 2D and unlike `DrawRectangle`.
- `DrawCylinder` takes the **bottom** centre, not the middle. Give it a top radius of 0 and you have a cone.
- `DrawPlane` is always flat on the XZ plane. It takes a **Vector2** size because a flat thing only has two dimensions.
- `DrawGrid` is centred on the origin, so a grid over a level that starts at 0 and grows positive will only cover a quarter of it.

11.2 Why everything looks flat

The first 3D scene anybody draws looks like coloured paper cut-outs. Every face of the cube is exactly the same shade, so the shape reads as a hexagon rather than a box.

That is not a bug. Plain `DrawCube` does no lighting at all — there is no light in the scene, so every surface just gets its flat colour. Real lighting means shaders, which is a much later chapter of your life.

Three cheap tricks get you 90% of the way, and the lessons use all three:

1. **Draw the wireframe on top.** A cube plus its dark edges reads as a solid object immediately:

```
DrawCube(p, 2, 2, 2, MAROON);
DrawCubeWires(p, 2, 2, 2, (Color){ 30, 26, 24, 255 });
```

2. **Vary the colour per object.** Identical walls merge into one mass; a little variation separates them. The mini Doom shades each wall cell from its grid coordinates:

```
int shade = 88 + ((col*7 + row*13) % 26);
```

3. **Make the floor and ceiling different.** Two `DrawPlane` calls in different colours do more for the sense of being inside a room than any amount of geometry.

11.3 A fake shadow

Nothing tells you where an object is in 3D like a shadow, and a convincing one costs one line: a very flat dark cube on the floor, directly under the object.

```
DrawCube((Vector3){ obj.x, 0.01f, obj.z }, 1.0f, 0.02f, 1.0f, Fade(BLACK, 0.5f));
```

The `0.01f` lifts it a hair above the floor. Put it at exactly 0 and it fights with the floor plane for the same pixels, which produces a horrible flickering called *z-fighting*. Whenever two surfaces occupy the same plane and shimmer, nudge one of them.

11.4 Making something face the player

Enemies in early first person games were flat images that always turned to face you. raylib does that with billboards:

```
DrawBillboard(camera, texture, position, size, WHITE);
```

If you have no texture, you can still get the effect with geometry. Work out the direction from the object to the camera, and place details along it — this is the trick the mini Doom uses to give its enemies eyes that follow you:

```
Vector3 toPlayer = Vector3Normalize(Vector3Subtract(camera.position, e->position));
Vector3 side = Vector3Normalize(Vector3CrossProduct(toPlayer, (Vector3){ 0, 1, 0 }));
Vector3 base = Vector3Add(e->position, Vector3Scale(toPlayer, 0.22f));
DrawSphere(Vector3Add(base, Vector3Scale(side, 0.10f)), 0.055f, YELLOW);
DrawSphere(Vector3Add(base, Vector3Scale(side, -0.10f)), 0.055f, YELLOW);
```

That is the first time we have used the cross product for something practical. Chapter 12 explains it properly; for now, read it as “give me the direction perpendicular to these two”.

11.5 Models, when you outgrow cubes

```
Model model = LoadModel("robot.glb");           // once, outside the loop
DrawModel(model, position, scale, WHITE);         // every frame
DrawModelEx(model, position, rotAxis, rotAngle, scaleV, WHITE);
UnloadModel(model);                               // once, after the loop
```

Same load-outside-unload-after rule as textures. raylib reads `.obj`, `.gltf`, `.glb`, `.iqm` and `.vox`, and `.glb` is the one to prefer because it packs the mesh, the materials and the textures into a single file.

11.6 The maths helpers

Vector maths lives in a separate header you have to include yourself:

```
#include "raymath.h"
```

The handful you will actually use:

Function	What it gives you
<code>Vector3Add(a, b)</code>	$a + b$
<code>Vector3Subtract(a, b)</code>	$a - b$: the vector from b to a
<code>Vector3Scale(v, f)</code>	v stretched by f
<code>Vector3Length(v)</code>	how long it is
<code>Vector3Normalize(v)</code>	same direction, length exactly 1
<code>Vector3Distance(a, b)</code>	distance between two points
<code>Vector3CrossProduct(a, b)</code>	a direction perpendicular to both

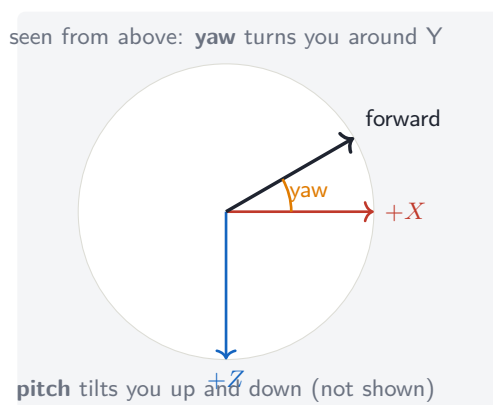
THE KEY IDEA

`Vector3Normalize` is the one to understand. A normalized vector has length 1, so it carries a *direction* and nothing else. Multiply it by a speed and you get a velocity; multiply it by a distance and you get an offset. Almost every movement line in the next chapters is `normalize(something) * speed * dt`.

Your own first person camera

This is the chapter people brace themselves for, and it is smaller than it looks. There are two angles and one formula, and then you are done.

12.1 The two angles



- **yaw** — the horizontal turn, spinning around the Y axis. Looking left and right.
- **pitch** — the vertical tilt, around the X axis. Looking up and down.

There is a third, *roll*, which tilts your head sideways. First person games almost never use it, so we will not either.

Both are stored in **radians**, because that is what C's `sinf` and `cosf` want. A full turn is 2π . Convert for display with raylib's `RAD2DEG` and `DEG2RAD` constants.

12.2 From angles to a direction

```
Vector3 forward = {
    cosf(pitch)*cosf(yaw),
    sinf(pitch),
    cosf(pitch)*sinf(yaw)
};
```

If trigonometry is a distant memory, here is the intuition and you can leave it at that. Ignore pitch for a second: on the flat ground, `cos(yaw)` and `sinf(yaw)` trace a circle of radius 1 as yaw goes round. That is the direction you are facing, on the floor.

Now add pitch. `sinf(pitch)` is how much you are looking up, and it goes straight into Y. `cosf(pitch)` shrinks the flat part: look straight up and nothing is left of the horizontal direction, which is exactly right.

Starting value: `yaw = -PI/2` points you along $-Z$, which is conventionally “into the screen”. `pitch = 0` is the horizon.

12.3 From a direction to sideways

To strafe you need the direction to your right. That is the **cross product** of where you are looking and where up is:

```
Vector3 right = Vector3Normalize(
    Vector3CrossProduct(forward, (Vector3){ 0.0f, 1.0f, 0.0f }));
```

All you need to know about the cross product: give it two directions and it hands back a third one perpendicular to both. Forward and up are perpendicular-ish, and the thing perpendicular to both of them is sideways. That is the entire reason it appears in every camera in existence.

12.4 Moving

```
Vector3 flatForward = Vector3Normalize((Vector3){ forward.x, 0.0f, forward.z });

Vector3 move = { 0 };
if (IsKeyDown(KEY_W)) move = Vector3Add(move, flatForward);
if (IsKeyDown(KEY_S)) move = Vector3Subtract(move, flatForward);
if (IsKeyDown(KEY_D)) move = Vector3Add(move, right);
if (IsKeyDown(KEY_A)) move = Vector3Subtract(move, right);

if (Vector3Length(move) > 0.0f) move = Vector3Normalize(move);

camera.position = Vector3Add(camera.position, Vector3Scale(move, speed*dt));
camera.target   = Vector3Add(camera.position, forward);
```

That last line is the one that ties it all together: the camera looks at a point one unit in front of the eye, in the direction we computed. Forget it and the camera moves but never turns.

12.5 The four details that make or break the feel

1. **Flatten the forward vector for walking.** Notice `flatForward` above: the Y component is thrown away and the result renormalized. Without it, looking at the sky and pressing W makes you fly.
2. **Clamp the pitch** to just under 90 degrees:

```
if (pitch > 1.55f) pitch = 1.55f;    // about 89 degrees
if (pitch < -1.55f) pitch = -1.55f;
```

At exactly 90, forward is parallel to up, the cross product collapses to zero, `right` becomes undefined and the camera snaps violently. This is *gimbal lock*, it is famous, and one `if` avoids it.

3. **Normalize the movement vector.** Hold W and D together without it and the two unit vectors add up to a length of $\sqrt{2}$: you move 41% faster diagonally. Players find that exploit within a minute.
4. **Multiply by `dt`.** As always.

12.6 Reading the mouse

```
DisableCursor();                                // once, at startup

Vector2 mouse = GetMouseDelta();
yaw    += mouse.x*0.003f;
pitch -= mouse.y*0.003f;                        // note the MINUS
```

Pitch subtracts because screen Y grows downwards while looking up is positive pitch — the 2D and 3D Y axes disagreeing one final time. Flip that sign and you have inverted mouse look, which some people actually prefer, so make it an option rather than a bug.

The 0.003f is mouse sensitivity, in radians per pixel. Expose it as a setting; everybody wants a different one.

12.7 The finishing touch: head bob

Pure decoration, three lines, and it is most of what makes movement feel physical:

```
if (walking) bobTimer += dt*speed*1.6f;
float bob = walking ? sinf(bobTimer)*0.045f : 0.0f;
camera.position.y = EYE_HEIGHT + bob;
```

A sine wave on the eye height, running faster when you run. Keep the amplitude tiny — 4 centimetres is plenty. Anything bigger and players feel seasick.

RUN IT

Lesson 4 of the 3D course is this chapter as a running program, printing yaw, pitch and the forward vector live so you can watch the numbers as you turn.

```
./course3d/build.sh 4
```

Collisions in 3D

13.1 The same idea, one more axis

The 3D equivalent of a **Rectangle** is an **AABB**: an *axis-aligned bounding box*. A box that never rotates, described by its two opposite corners.

```
typedef struct BoundingBox {
    Vector3 min;    // the corner with the smaller x, y and z
    Vector3 max;    // the corner with the larger x, y and z
} BoundingBox;
```

And the rule is chapter 6’s rule with a third line added: two boxes collide if they overlap on **all three** axes at once.

```
overlapX = (a.min.x <= b.max.x) && (a.max.x >= b.min.x);
overlapY = (a.min.y <= b.max.y) && (a.max.y >= b.min.y);
overlapZ = (a.min.z <= b.max.z) && (a.max.z >= b.min.z);
hit      = overlapX && overlapY && overlapZ;
```

raylib gives you it as `CheckCollisionBoxes(a, b)`. Its relatives:

```
CheckCollisionBoxes(boxA, boxB);
CheckCollisionSpheres(centreA, rA, centreB, rB);
CheckCollisionBoxSphere(box, centre, radius);
```

13.2 Centre and size, not corners

Nobody stores their objects as two corners. You store a position and a size, and convert when you need to test:

```
BoundingBox BoxFromCenter(Vector3 centre, Vector3 size)
{
    BoundingBox b;
    b.min = (Vector3){ centre.x - size.x/2, centre.y - size.y/2, centre.z - size.z/2 };
    b.max = (Vector3){ centre.x + size.x/2, centre.y + size.y/2, centre.z + size.z/2 };
    return b;
}
```

You will write this function in every 3D project you ever start. It is the same “subtract half the size” from chapter 3, with a z on the end.

WATCH OUT

The player’s `camera.position` is the height of their **eyes**, not their middle. A 1.7 m person standing on the floor has eyes at $Y = 1.7$ and a body centred at $Y = 0.85$. Build the collision box around the body, not the eyes, or your player will collide with things at head height and walk through everything below the knee.

```
Vector3 bodyCentre = { position.x, position.y - EYE_HEIGHT/2.0f, position.z };
Vector3 bodySize   = { RADIUS*2, EYE_HEIGHT, RADIUS*2 };
```

13.3 Resolving, per axis, exactly as in 2D

```
Vector3 attempt = camera.position;
attempt.x += dx;
if (Collides(attempt)) { /* refused: keep the old x */ }
else camera.position.x = attempt.x;

attempt = camera.position;
attempt.z += dz;
if (Collides(attempt)) { /* refused: keep the old z */ }
else camera.position.z = attempt.z;
```

X then Z, because on foot those are the two ground axes. If you add jumping and gravity, Y becomes a third pass with the same shape — and the Y pass is where you discover you have landed, because a refused downward move means there is floor under you.

THE KEY IDEA

Detect with one call, resolve one axis at a time. It was true in 2D, it is true in 3D, and it will still be true in whatever engine you use next.

13.4 Debugging: draw the boxes

```
DrawBoundingBox(playerBox, GREEN);
DrawBoundingBox(wallBox, hit ? RED : GREEN);
```

Nine times out of ten, a collision bug is not a logic bug — the box is simply not where you think it is. It is offset by half a height, or it is built from last frame's position, or it is twice the size of the thing you drew. Draw it and the bug is visible in a second instead of an hour.

13.5 Grids: the other way, and the fast one

Testing against a list of boxes is fine for a few dozen walls. For a whole level there is a better approach, and it is what the original Doom did: store the level as a **grid**, and convert a position straight into a cell index.

```
static bool IsWall(float x, float z)
{
    int col = (int)floorf(x/CELL + 0.5f);
    int row = (int)floorf(z/CELL + 0.5f);

    if (col < 0 || col >= MAP_COLS || row < 0 || row >= MAP_ROWS) return true;

    return (MAP[row][col] == '#');
}
```

No loop. It costs the same whether the map has 10 walls or 10,000, and the out-of-bounds check doubles as an invisible wall around the level.

To give the player some width, test the four corners of the square around them instead of the single centre point:

```
static bool CanStandAt(float x, float z)
{
    const float r = PLAYER_RADIUS;
    return !IsWall(x - r, z - r) && !IsWall(x + r, z - r)
        && !IsWall(x - r, z + r) && !IsWall(x + r, z + r);
}
```

13.6 Shooting: rays

A **Ray** is a starting point and a direction:

```
typedef struct Ray { Vector3 position; Vector3 direction; } Ray;
```

Fire one out of the camera and ask what it meets:

```
Ray shot = { camera.position, forward };
RayCollision hit = GetRayCollisionBox(shot, EnemyBox(enemy.position));

if (hit.hit) { /* hit.distance, hit.point and hit.normal are filled in */ }
```

For a mouse click rather than a crosshair, build the ray from the cursor:

```
Ray pick = GetScreenToWorldRay(GetMousePosition(), camera);
```

WATCH OUT

A ray goes on forever and hits everything in its path. To find what you actually shot, loop over the candidates, keep the one with the smallest `hit.distance`, and compare it against the distance to the nearest wall. Skip that last comparison and your players will happily shoot enemies through solid walls.

RUN IT

Lesson 5 shows one box turning red on contact, with the three axis overlaps printed separately; lesson 6 is a closed room where the walls actually stop you.

```
./course3d/build.sh 5    and    ./course3d/build.sh 6
```

Building a silly little Doom

Everything so far, assembled. The finished program is `course3d/lesson07_mini_doom.c`: a corridor maze you walk through, enemies that come after you, a gun that shoots them, and a HUD with a minimap. About 350 lines, and you have already met every idea in it.

RUN IT

Run it first, then read this chapter with the source open next to you.

```
./course3d/build.sh 7
```

14.1 The level is a piece of text

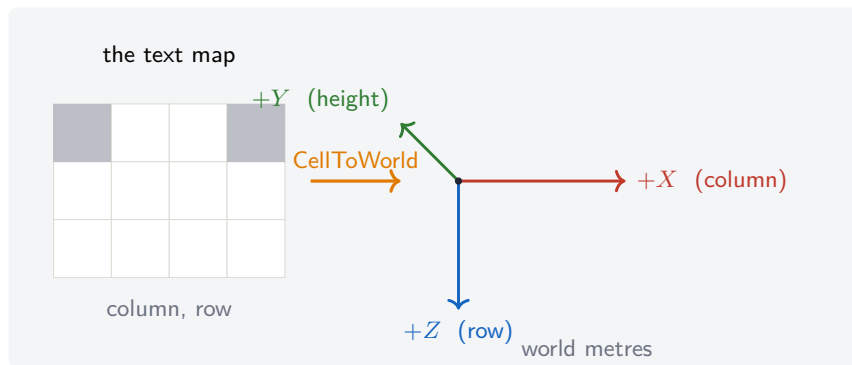
```
static const char *MAP[MAP_ROWS] = {
    "#####",
    "#S....#.....#",
    "#.....#...E...#",
    "#.###.#.####.#",
    ...
};
```

is a wall, . is floor, S is where the player starts and E is an enemy. Designing a level is now typing characters into a text editor, and you can redesign the whole game in thirty seconds. Plenty of shipped games stored their levels exactly like this.

14.2 From map cell to world position

The map is a grid of rows and columns. The world is metres along X, Y and Z. The bridge between them is one function, and it is worth staring at until it is obvious:

```
static Vector3 CellToWorld(int col, int row, float y)
{
    return (Vector3){ col*CELL, y, row*CELL };
}
```



THE KEY IDEA

Map **column** becomes world **X**. Map **row** becomes world **Z**. **Y** is not in the map at all, because a top-down plan has no height in it. This one mapping is the whole relationship between a 2D level editor and a 3D world, and it runs backwards too — that is how the minimap turns a 3D position back into a dot.

14.3 Reading the map once, at startup

`ResetGame()` walks the text and turns markers into game state:

```
for (int row = 0; row < MAP_ROWS; row++)
    for (int col = 0; col < MAP_COLS; col++)
    {
        if (MAP[row][col] == 'S')
            g->camera.position = CellToWorld(col, row, EYE_HEIGHT);
        else if (MAP[row][col] == 'E' && g->enemyCount < MAX_ENEMIES)
        {
            g->enemies[g->enemyCount].position = CellToWorld(col, row, 0.9f);
            g->enemies[g->enemyCount].alive = true;
            g->enemyCount++;
        }
    }
}
```

Because all the state lives in one `Game` struct, restarting the level is a single call to `ResetGame(&game)` bound to the R key. Worth copying as a habit: put the game state in a struct, write a reset function, and you get restarts for free.

14.4 The order of the frame

The loop is one long sequence, and the order matters:

1. **Look** — mouse delta into yaw and pitch, pitch clamped, then build forward, right and flatForward (chapter 12).
2. **Move** — keys into a direction, normalized, times speed times `dt`, resolved one axis at a time against the grid (chapter 13).
3. **Shoot** — on click, fire a ray and see what it finds.
4. **Enemies** — walk towards the player, hurt them on contact.
5. **Draw the world** — inside `BeginMode3D`.

6. Draw the HUD — after `EndMode3D`, in screen pixels.

Notice that all six steps read the same `forward` vector that step 1 computed. Compute it once per frame, near the top, and pass it around. Computing it twice with different values is how you get a gun that shoots slightly to the left of the crosshair.

14.5 Walls

One cube per # cell, with a shade that varies by position so the corridors do not look like a single flat mass:

```
Vector3 centre = CellToWorld(col, row, WALL_HEIGHT/2);
int shade = 88 + ((col*7 + row*13) % 26);
DrawCube(centre, CELL, WALL_HEIGHT, CELL, wall);
DrawCubeWires(centre, CELL, WALL_HEIGHT, CELL, (Color){ 30, 26, 24, 255 });
```

Note `WALL_HEIGHT/2` for the Y: the cube's centre is at half its height, so it sits on the floor instead of sinking into it (chapter 9).

Floor and ceiling are two `DrawPlane` calls in different colours. Two lines, and suddenly you are indoors.

14.6 Shooting

```
Ray shot = { game.camera.position, forward };

// how far is the wall in front of us?
float wallDistance = 100.0f;
for (float t = 0.1f; t < 60.0f; t += 0.1f)
{
    Vector3 p = Vector3Add(shot.position, Vector3Scale(shot.direction, t));
    if (IsWall(p.x, p.z)) { wallDistance = t; break; }
}

// the nearest enemy closer than that wall
int target = -1;
float best = wallDistance;

for (int i = 0; i < game.enemyCount; i++)
{
    if (!game.enemies[i].alive) continue;
    RayCollision hit = GetRayCollisionBox(shot, EnemyBox(game.enemies[i].position));
    if (hit.hit && hit.distance < best) { best = hit.distance; target = i; }
}
```

Two things are happening. The loop marching along the ray in 10 cm steps is a crude *raycast* against the grid — it walks forward until it is inside a wall cell, and that is your line of sight. Seeding `best` with that distance is what stops you shooting through walls: an enemy only counts if it is nearer than the wall.

WATCH OUT

Stepping along a ray in fixed increments can step straight over a thin wall if the step is bigger than the wall. Ours is 10 cm against 2 m walls, so it is safe. Proper raycasters use a grid traversal algorithm (look up “DDA”) that cannot miss, and that is the next thing to learn if you want to go further down this road.

14.7 Enemies

The simplest AI that still feels like a game: if you can see me and I am not touching you, walk at you.

```
Vector3 toPlayer = Vector3Subtract(game.camera.position, e->position);
toPlayer.y = 0.0f; // ignore height: they walk, not fly
float distance = Vector3Length(toPlayer);

if (distance < 12.0f && distance > 0.9f)
{
    Vector3 step = Vector3Scale(Vector3Normalize(toPlayer), 1.6f*dt);
    if (CanStandAt(e->position.x + step.x, e->position.z)) e->position.x += step.x;
    if (CanStandAt(e->position.x, e->position.z + step.z)) e->position.z += step.z;
}
else if (distance <= 0.9f)
{
    game.health -= 25.0f*dt; // four seconds of contact kills you
}
```

The pattern `normalize(target - me) * speed * dt` is how absolutely everything chases everything else: enemies, homing missiles, a pet following you. Learn it once, use it forever.

Their movement is resolved per axis too — same two lines as the player — so they slide along walls instead of getting stuck in corners.

14.8 The HUD, or where the 2D course pays off

Everything after `EndMode3D()` is plain screen pixels:

- **Crosshair** — four short `DrawLine` calls with a gap in the middle, centred on `GetScreenWidth()/2`.
- **The gun** — three rectangles at the bottom of the screen, offset by the same `sinf(bobTimer)` that bobs the camera, so the weapon sways with your steps. A muzzle flash is two circles drawn for 0.06 seconds after a shot.
- **Health bar** — a background rectangle and a second one whose width is `224.0f*health/100.0f`, turning red below 30.
- **Minimap** — the text map drawn as little squares, with the player and enemies as dots. World X divided by the cell size gives the map column, which gives the screen X. Chapter 9's "the floor is XZ" used in reverse.

```
int px = ox + (int)(game.camera.position.x/CELL*tile);
int py = oy + (int)(game.camera.position.z/CELL*tile);
DrawCircle(px, py, 3, SKYBLUE);
DrawLine(px, py, px + (int)(forward.x*14), py + (int)(forward.z*14), SKYBLUE);
```

That last line is the little arrow showing which way you face: the 3D forward vector, with its Y thrown away, drawn as a 2D line. Three dimensions and two dimensions, in the same frame, sharing one number.

14.9 Things to change first

The point of having the source is breaking it. In rough order of fun per minute:

-
1. Redraw the map. Add a room, a dead end, a long corridor.
 2. Change `WALL_HEIGHT` to 8 and walk around. Then try 1.5.
 3. Set `fovy` to 110 and then to 40. Feel how much the lens changes the game.
 4. Give the enemies different speeds, or make them stop when they are far away and only wake when they see you.
 5. Add a pickup: a `P` in the map, drawn as a spinning cube (rotate it with `GetTime()`), that refills ammo when you touch it. Everything you need is in chapters 9 and 13.
 6. Add jumping: a `velocityY`, gravity subtracted every frame, and the Y axis resolved like X and Z.

Part II

The rest of the library

Sound and music

Of everything left in the library, this is the one that changes how finished your game feels for the least work. Three function calls and the mini Doom stops being a tech demo.

15.1 One line before anything

```
InitAudioDevice();    // right after InitWindow
...
CloseAudioDevice();   // right before CloseWindow
```

Nothing in raudio works before that call, exactly like `InitWindow` for graphics. If sounds silently do nothing, check this first.

15.2 The four things raudio gives you

Type	Lives	Use it for
Wave	in RAM, raw samples	building or editing audio yourself
Sound	in RAM, decoded	short clips: gunshots, steps, clicks
Music	streamed from disk	long tracks: the soundtrack
AudioStream	you push samples	synthesisers, anything procedural

The split between **Sound** and **Music** is about memory. A three minute track decoded into RAM is around 30 MB; streamed, it costs a few kilobytes of buffer. A footstep is 4 KB either way, so it may as well be resident and instant.

15.3 Sounds

```
Sound shot = LoadSound("shot.wav");    // once, outside the loop

if (IsMouseButtonPressed(MOUSE_BUTTON_LEFT)) PlaySound(shot);

UnloadSound(shot);                      // once, after the loop
```

Same load-outside, unload-after rule as textures. raylib reads `.wav`, `.ogg`, `.mp3`, `.flac`, `.qoa` and a few more.

`PlaySound` does not wait and does not queue — it fires and returns immediately. Call it twice in one frame and you get two overlapping copies, which is exactly what you want for gunfire.

15.3.1 The trick that makes repeated sounds bearable

A sound effect played identically fifty times in ten seconds becomes torture. Real games vary it slightly on every shot:

```
SetSoundPitch(shot, 0.85f + GetRandomValue(0, 30)/100.0f);
SetSoundPan(shot, GetRandomValue(20, 80)/100.0f);
PlaySound(shot);
```

Pitch is a multiplier, so 1.0 is unchanged and 0.5 is an octave down. Pan runs 0.0 (left) to 1.0 (right), with 0.5 centred.

WATCH OUT

`SetSoundPitch` changes that `Sound` object for every future play, not just the next one. If you want two enemies to sound different at the same time you need `LoadSoundAlias()`, which shares the samples but gives you a second set of playback settings, and costs almost nothing.

15.4 Music

```
Music track = LoadMusicStream("theme.ogg");
track.looping = true;
PlayMusicStream(track);

while (!WindowShouldClose())
{
    UpdateMusicStream(track);    // <-- EVERY FRAME. Not optional.
    ...
}

UnloadMusicStream(track);
```

WATCH OUT

`UpdateMusicStream()` is the single most forgotten call in raylib. It refills the streaming buffer from disk. Miss it and your music plays for a fraction of a second and stops — no error, no warning, just silence. If music “does not work”, this is the reason nine times out of ten.

Pause, resume and seek behave as you would expect, and two getters let you draw a progress bar:

```
PauseMusicStream(track); ResumeMusicStream(track); StopMusicStream(track);
SeekMusicStream(track, 30.0f);                      // jump to 30 seconds

float played = GetMusicTimePlayed(track);
float length = GetMusicTimeLength(track);
```

15.5 Making sounds with no sound files

You do not need an artist to get started. A `Wave` is just numbers, and a 16-bit mono wave is one `short` per sample, from -32768 to 32767 :

```
Wave wave = { 0 };
wave.sampleRate = 44100;      // samples per second
wave.sampleSize = 16;        // bits per sample
wave.channels = 1;           // mono
wave.frameCount = 44100/5;    // a fifth of a second

short *samples = (short *)MemAlloc(wave.frameCount*sizeof(short));

float phase = 0.0f;
for (unsigned int i = 0; i < wave.frameCount; i++)
{
    float t = (float)i/wave.frameCount;      // 0..1 through the sound
    float hz = 1400.0f + (200.0f - 1400.0f)*t; // pitch slides DOWN
    phase += 2.0f*PI*hz/44100.0f;

    float envelope = (1.0f - t)*(1.0f - t);   // fade out, or it clicks
    samples[i] = (short)(sinf(phase)*envelope*22000.0f);
}

wave.data = samples;
Sound laser = LoadSoundFromWave(wave);
```

Three ideas are doing all the work there, and they are the whole of retro sound design:

- **A sine wave** gives you a pure tone. `phase` accumulates and `sinf(phase)` oscillates.
- **Sliding the pitch** turns a beep into a laser (down) or a power-up (up).
- **An envelope** — multiplying by a value that falls to zero — makes it a sound instead of a rude noise. Without a fade-out, the sound stops mid-oscillation and the speaker cone jumps, which you hear as a click.

Mix in random noise instead of a sine and you have an explosion. That is literally what the **rFXGen** tool on raylib.com does, with sliders.

WATCH OUT

Use `MemAlloc()` rather than `malloc()` for wave data. `UnloadWave()` frees it with raylib's allocator, and mixing allocators is the kind of bug that crashes only on some machines.

15.5.1 Wave and Sound are two separate allocations

`LoadSoundFromWave` copies the samples to the audio device. The original `Wave` is still yours, and still needs freeing:

```
UnloadSound(laser);
UnloadWave(wave);      // both, not one
```

You can also write it out — `ExportWave(wave, "laser.wav")` — which is a genuinely useful way to build a sound in code, listen to it, and then ship the file.

15.6 Audio streams, for sound you generate live

When the samples cannot exist ahead of time — a synthesiser, an engine note that follows the revs, voice chat — you push them yourself:

```
AudioStream stream = LoadAudioStream(44100, 16, 1);
PlayAudioStream(stream);

// in the loop:
if (IsAudioStreamProcessed(stream))    // the device wants more
{
    static short chunk[4096];
    for (int i = 0; i < 4096; i++) { phase += step; chunk[i] = ...; }
    UpdateAudioStream(stream, chunk, 4096);
}
```

The contract is: ask whether a buffer has been consumed, and if so, hand over the next one. Miss the window and you hear a gap.

15.7 Volume, and being polite

```
SetMasterVolume(0.6f);           // everything
SetSoundVolume(shot, 0.4f);      // one sound
SetMusicVolume(track, 0.5f);     // one track
```

Ship with the master volume somewhere around 0.5 and put it in an options menu. Players will forgive almost anything except a game that is deafening on the first launch.

RUN IT

Module 9 synthesises a beep, a laser and an explosion in C, writes a four second tune to a real .wav and streams it back, and lets you play a live tone with the mouse controlling the pitch. No audio files involved.

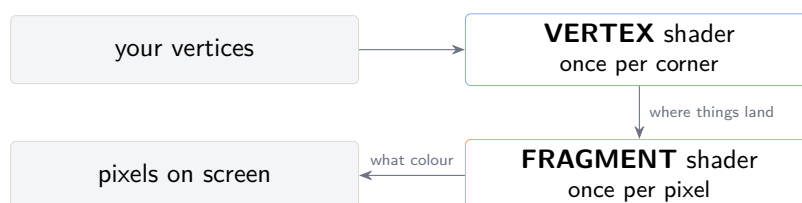
```
./cheatsheet/build.sh 9
```

Shaders, and why your cubes look flat

Chapter 11 told you that plain `DrawCube` does no lighting, and left it there. This is the chapter that explains what to do about it.

16.1 What a shader actually is

A shader is a small program that runs **on the GPU**, in parallel, an absurd number of times per frame. You write it in GLSL, a C-like language, and you get two of them:



- The **vertex** shader runs once per corner of your geometry and decides *where* it lands on screen.
- The **fragment** shader runs once per pixel and decides *what colour* that pixel is.

At 1080p a fullscreen effect runs your fragment shader two million times, sixty times a second. That is why it must be tiny and why it cannot talk back to your C code.

THE KEY IDEA

raylib ships a default pair that just draws things normally. You almost always write only the fragment shader and pass `NULL` for the vertex one. Lighting, colour grading, outlines, water, fog — all fragment shaders.

16.2 The three steps, which never change

```

// 1. LOAD, once, outside the loop
Shader shader = LoadShader(NULL, "glow.fs");           // from files
Shader shader = LoadShaderFromMemory(NULL, fsCode);    // from a string

// 2. FIND THE SLOTS, once: a lookup by name into the compiled program
int locTime = GetShaderLocation(shader, "uTime");

// 3. FEED AND USE, every frame
SetShaderValue(shader, locTime, &time, SHADER_UNIFORM_FLOAT);

BeginShaderMode(shader);
    DrawTexture(tex, 0, 0, WHITE);    // drawn THROUGH the shader
EndShaderMode();
  
```

That `Begin/End` pair is the same contract as the cameras: inside it, the shader applies; outside, normal drawing. Your HUD goes outside, for the same reason as always.

16.3 Uniforms: the only way to talk to the GPU

A *uniform* is a variable you send from C into the shader. It is called uniform because it has the same value for every pixel in that draw call.

C side	GLSL side
<code>SHADER_UNIFORM_FLOAT</code>	<code>uniform float uTime;</code>
<code>SHADER_UNIFORM_VEC2</code>	<code>uniform vec2 uLight;</code>
<code>SHADER_UNIFORM_VEC3</code>	<code>uniform vec3 uColour;</code>
<code>SHADER_UNIFORM_INT</code>	<code>uniform int uMode;</code>
<code>SetShaderValueTexture</code>	<code>uniform sampler2D uSecond;</code>

WATCH OUT

`GetShaderLocation` returns `-1` if the name is not found — and GLSL compilers *delete unused uniforms*, so a uniform you declared but never read in the shader body will not be there. A silent `-1` that makes `SetShaderValue` do nothing is the classic first shader bug.

16.4 A fragment shader, line by line

```
#version 330

in vec2 fragTexCoord;           // from the vertex shader: where we are, 0..1
in vec4 fragColor;              // the tint passed to DrawTexture

uniform sampler2D texture0;      // the texture raylib binds automatically
uniform vec4 colDiffuse;         // raylib's diffuse colour

out vec4 finalColor;            // what we must write

void main()
{
    vec4 texel = texture(texture0, fragTexCoord)*colDiffuse*fragColor;
    float grey = dot(texel.rgb, vec3(0.299, 0.587, 0.114));
    finalColor = vec4(grey, grey, grey, texel.a);
}
```

Four things to notice:

1. `#version 330` must be the very first thing in the file or string. Not a comment, not a blank line. On desktop raylib uses GLSL 330.
2. `texture0`, `colDiffuse`, `fragTexCoord` and `fragColor` are names raylib fills in for you. Spell them exactly or you get nothing.
3. `vec4` is four floats, and colours here run **0.0 to 1.0**, not 0 to 255.
4. The last line of the function must write `finalColor`. That is the shader's entire output.

Those weights (0.299, 0.587, 0.114) are how bright each channel looks to a human eye — green far more than blue. Averaging the three instead gives a muddy grey, which is why this particular triple turns up everywhere.

16.5 Errors show up at run time, not compile time

Your C compiles fine whatever nonsense is in the shader string: the GPU compiler only runs when `LoadShader` executes. Watch the terminal:

```
INFO: SHADER: [ID 5] Fragment shader compiled successfully
INFO: SHADER: [ID 6] Program shader loaded successfully
```

If instead you see `SHADER: [ID 5] Failed to compile fragment shader`, the line number in the message refers to the *shader* source. A failed shader falls back to the default one, so your scene keeps rendering normally and the only sign of trouble is that your effect does nothing.

16.6 Lighting, honestly

Real lighting is a fragment shader that receives the surface normal and the light positions, and computes how much each light hits each pixel. `raylib` does not build that in — the `examples/shaders/` folder has it, in `shaders_basic_lighting.c`, as about 80 lines of GLSL.

Before you go there, be aware of the cheaper options from chapter 11: wireframes over solid shapes, colour variation per object, and a distinct floor and ceiling. They cost nothing, they work everywhere, and plenty of good-looking games never do anything more.

THE KEY IDEA

Learn shaders when flat colour is actually holding your game back, not before. It is the deepest rabbit hole in graphics and the easiest place to spend three weeks and ship nothing.

RUN IT

Module 3 has three fragment shaders written as C strings — greyscale, a wave distortion driven by a `uTime` uniform, and a spotlight that follows your mouse. The HUD is drawn outside the pair so you can see what the shader does and does not touch.

```
./cheatsheet/build.sh 3
```

Meshes, models and materials

Chapter 11 drew everything with `DrawCube` and friends. Those are throwaways: they build geometry from scratch every frame and cannot carry a texture. This chapter is the step up.

17.1 The three layers

Mesh	raw geometry: vertex positions, normals, texture coordinates. Lives on the GPU. No colour, no texture.
Material	how a surface looks: a shader plus a set of texture maps (diffuse, normal, roughness...) and a tint.
Model	one or more meshes, one or more materials, and a transform matrix tying them together.

A model loaded from a `.glb` file arrives with all three filled in. When you build one yourself, you assemble them:

```
Mesh mesh    = GenMeshCube(2.0f, 2.0f, 2.0f);
Model model = LoadModelFromMesh(mesh);           // wraps it, default white material
```

17.2 Generating geometry

You do not need a modelling program to get real meshes:

<code>GenMeshCube(w, h, l)</code>	a box
<code>GenMeshSphere(radius, rings, slices)</code>	more rings = smoother, slower
<code>GenMeshPlane(w, l, resX, resZ)</code>	a floor; resolution matters if you deform it
<code>GenMeshCylinder(radius, height, slices)</code>	pillars, trees, barrels
<code>GenMeshTorus(radius, size, rad, sides)</code>	a doughnut
<code>GenMeshKnot(...)</code>	a torus knot, for showing off
<code>GenMeshHeightmap(image, size)</code>	terrain from a greyscale image
<code>GenMeshCubicmap(image, size)</code>	a 3D maze from a pixel map

Those last two deserve attention. `GenMeshHeightmap` turns a greyscale picture into hills, where brightness is altitude. `GenMeshCubicmap` turns a black-and-white pixel map into walls — it is the mini Doom's text map, except it produces a single mesh instead of hundreds of separate cubes, so it draws in one call instead of two hundred.

17.3 Putting a texture on something

This is the thing `DrawCube` cannot do:

```
Texture2D tex = LoadTexture("wall.png");
model.materials[0].maps[MATERIAL_MAP_DIFFUSE].texture = tex;
```

Read it inside out: the model's first material, its diffuse map, its texture. `MATERIAL_MAP_DIFFUSE` is the plain colour map; there are slots for normal maps, roughness, occlusion and the rest, which only matter once you have a shader that reads them.

You can tint per draw as well, and the two multiply:

```
DrawModel(model, position, 1.0f, WHITE);    // texture as-is
DrawModel(model, position, 1.0f, RED);      // texture tinted red
```

17.4 Drawing

```
DrawModel(model, position, scale, tint);
DrawModelEx(model, position, rotationAxis, rotationAngle, scaleVec3, tint);
DrawModelWires(model, position, scale, tint);
DrawBoundingBox(GetMeshBoundingBox(mesh), GREEN);
```

`DrawModelEx` is the one you want for anything that turns: an axis, an angle in degrees, and a per-axis scale.

```
DrawModelEx(model, pos, (Vector3){ 0, 1, 0 }, angle, (Vector3){ 1, 1, 1 }, WHITE);
```

Rotating around (0,1,0) means spinning about the vertical axis, which is what you want for a character turning or a pickup idling.

17.5 Loading real models

```
Model robot = LoadModel("robot.glb");    // once
DrawModel(robot, position, 1.0f, WHITE);  // every frame
UnloadModel(robot);                       // once
```

raylib reads `.obj`, `.gltf`, `.glb`, `.iqm`, `.m3d` and `.vox`. Prefer `.glb`: it packs the mesh, the materials and the textures into one file, so there is nothing to go missing and no paths to fix.

WATCH OUT

`UnloadModel` frees the meshes it owns but deliberately does *not* free textures or shaders — you might be sharing them between models, so raylib leaves them to you. Unload your textures yourself, exactly once.

17.6 Animation

```
int animCount = 0;
ModelAnimation *anims = LoadModelAnimations("robot.glb", &animCount);
int frame = 0;
```

```
// in the loop:
frame = (frame + 1) % anims[0].frameCount;
UpdateModelAnimation(robot, anims[0], frame);
DrawModel(robot, position, 1.0f, WHITE);

UnloadModelAnimations(anims, animCount);
```

The model must have been exported with a skeleton, `.glb` and `.iqm` being the usual choices. Note that stepping `frame` once per frame ties the animation to your frame rate — use a timer, as in chapter 5, if you care.

17.7 Billboards: the old trick, still useful

A billboard is a flat image that always turns to face the camera. Every enemy in Doom was one.

```
DrawBillboard(camera, texture, position, size, WHITE);
DrawBillboardRec(camera, texture, sourceRec, position, size, WHITE); // one sprite of a
sheet
```

They are cheap, they look deliberately retro, and `DrawBillboardRec` lets you animate them from a sprite sheet exactly as in chapter 7. For particles, smoke, flames and pickups they are still the right answer today.

17.8 When to stop using DrawCube

Stay with immediate shapes if	Move to models when
prototyping	you want textures
a handful of objects	you want lighting via a shader
debug visualisation	you draw hundreds of the same thing
flat colour is fine	you need animation

And when you are drawing the same mesh thousands of times — grass, crowds, asteroids — there is `DrawMeshInstanced`, which sends one mesh and an array of transforms in a single call. It is the difference between 50 FPS and 2000.

RUN IT

Module 8 generates a cube, a sphere and a torus, puts a generated checkerboard on the cube's material, draws a billboard, and lets you click any object to pick it with a ray. TAB switches between models, immediate shapes and wireframe so you can compare them.

```
./cheatsheet/build.sh 8
```

Images, render textures and colour

Chapter 7 covered enough of rtextures to draw a sprite. This is the rest of the module: editing pixels, drawing into textures, and the colour helpers.

18.1 Image work happens on the CPU

An `Image` is pixels in ordinary RAM, and raylib gives you a small image editor's worth of functions to work on them:

<code>ImageResize / ImageResizeNN</code>	smooth, or nearest-neighbour for pixel art
<code>ImageCrop / ImageAlphaCrop</code>	cut a region, or trim transparent edges
<code>ImageFlipHorizontal / Vertical</code>	mirror
<code>ImageRotate / ImageRotateCW</code>	turn
<code>ImageColorTint / Invert / Grayscale</code>	recolour every pixel
<code>ImageColorBrightness / Contrast</code>	adjust
<code>ImageColorReplace</code>	swap one exact colour for another
<code>ImageDrawRectangle / Circle / Text</code>	draw into the image
<code>ImageDraw</code>	paste one image into another
<code>ImageFormat</code>	convert the pixel layout

WATCH OUT

Almost all of these take a `Image *` and modify it in place — `ImageResize(&img, 128, 128)`, not `img = ImageResize(...)`. And they all work on the CPU, so they are far too slow to run every frame. Do them once at load time, then upload.

Use `ImageCopy` before editing if you want to keep the original.

18.2 Generating images from nothing

```
GenImageColor(w, h, colour);           // a flat fill
GenImageGradientLinear(w, h, angle, a, b); // a linear gradient
GenImageGradientRadial(w, h, density, a, b); // a radial one: glows, vignettes
GenImageChecked(w, h, cw, ch, a, b);    // a checkerboard
GenImageWhiteNoise(w, h, factor);       // static
GenImagePerlinNoise(w, h, offX, offY, scale); // smooth noise: terrain, clouds
GenImageCellular(w, h, tileSize);       // organic cell patterns
```

This is how you get a game looking like something before an artist exists. `GenImagePerlinNoise` feeding `GenMeshHeightmap` from the last chapter gives you an entire landscape in four lines.

18.3 Render textures: drawing into a texture

A `RenderTarget2D` is a texture you can draw into, with its own `Begin/End` pair:

```
RenderTarget2D canvas = LoadRenderTarget(320, 240);    // once

// in the loop, BEFORE BeginDrawing:
BeginTextureMode(canvas);
    ClearBackground(BLACK);
    DrawCircle(160, 120, 40, ORANGE);
EndTextureMode();

BeginDrawing();
    DrawTexture(canvas.texture, 0, 0, WHITE);
EndDrawing();

UnloadRenderTarget(canvas);                            // once
```

This unlocks four things you cannot do otherwise:

1. **Pixel-art at a fixed resolution.** Render your game at 320×180 and scale it up to the window. Every pixel stays square and crisp, at any window size.
2. **A real minimap.** Draw the world from a top-down camera into a texture, then draw that texture in the corner.
3. **Fullscreen shader effects.** Render the scene into a texture, then draw that texture through a shader. Bloom, colour grading, blur — all of it works this way.
4. **In-world screens.** Security monitors, mirrors, portals.

WATCH OUT

A render texture comes out **upside down**, because OpenGL's Y axis points the other way. The fix is a negative height in the source rectangle:

```
0, 0, (float)rt.texture.width, -(float)rt.texture.height (Rectangle)
```

Use `DrawTexturePro` or `DrawTextureRec` with that, and never plain `DrawTexture`, or your scene is mirrored vertically.

18.4 Texture filters and mipmaps

```
SetTextureFilter(tex, TEXTURE_FILTER_POINT);           // crisp: pixel art
SetTextureFilter(tex, TEXTURE_FILTER_BILINEAR);        // smooth: photos, 3D
GenTextureMipmaps(&tex);
SetTextureFilter(tex, TEXTURE_FILTER_TRILINEAR);       // smooth + mipmaps
```

`POINT` is what makes pixel art look like pixel art; the default bilinear filter will blur your beautiful 16×16 sprite into mush the moment you scale it.

Mipmaps are pre-shrunk copies of a texture. Without them, a wall texture seen from far away shimmers as you walk, because each pixel is sampling a random spot in the full-size image. Generate them for anything in a 3D world; skip them for 2D sprites drawn at their real size.

18.5 Colour helpers

<code>Fade(colour, 0.5f)</code>	same colour, half alpha
<code>ColorTint(colour, tint)</code>	multiply two colours
<code>ColorBrightness(colour, -0.4f)</code>	-1 black, +1 white
<code>ColorContrast(colour, 0.5f)</code>	
<code>ColorAlphaBlend(dst, src, tint)</code>	blend as if drawn on top
<code>ColorLerp(a, b, t)</code>	blend two colours
<code>ColorToHSV / ColorFromHSV</code>	hue, saturation, value
<code>ColorToInt / GetColor</code>	to and from 0xRRGGBBAA

HSV is the one worth knowing about. Shifting only the hue gives you a whole palette of related colours — team colours, damage states, day and night — from a single base:

```
Vector3 hsv = ColorToHSV(base);
Color enemyTeam = ColorFromHSV(hsv.x + 180, hsv.y, hsv.z); // opposite hue
```

And `GetColor(0x1F2430FF)` lets you paste hex values straight from a design tool, which is more pleasant than counting bytes.

RUN IT

Module 6 shows all six image generators, a chain of CPU edits, the four ways to draw a texture, a live render texture with its flipped source rectangle, and the colour helpers side by side.

```
./cheatsheet/build.sh 6
```

Fonts and text

`DrawText` with the built-in font gets you through a prototype. The moment you want a game that looks designed, you need the rest of `rtext`.

19.1 Loading a font properly

```
Font font = LoadFont("kenney.ttf");           // default size 32
Font font = LoadFontEx("kenney.ttf", 48, NULL, 0); // the one you want
```

`LoadFontEx` takes the size to rasterise at, and optionally a list of codepoints to include. Passing `NULL, 0` gives you the default set of 95 ASCII characters.

WATCH OUT

A font is baked into a bitmap at **one fixed size**. Load it at 16 and draw it at 64 and it will be blurry — that is not a bug, it is a small image being stretched. Load it at, or slightly above, the largest size you will actually draw.

For accented characters, and for anything beyond English, you must say so at load time:

```
int codepoints[512] = { 0 };
for (int i = 0; i < 95; i++) codepoints[i] = 32 + i;           // basic ASCII
for (int i = 0; i < 96; i++) codepoints[95 + i] = 0xC0 + i; // Latin-1 supplement

Font font = LoadFontEx("font.ttf", 32, codepoints, 191);
```

A character you did not ask for simply does not render. If your accents come out as blanks or question marks, this is why.

19.2 Drawing with it

```
DrawText(text, x, y, size, colour);           // default font only
DrawTextEx(font, text, position, size, spacing, colour); // any font
DrawTextPro(font, text, position, origin, rot, size, spacing, colour);
```

`DrawTextEx` is the everyday one. The `spacing` parameter is extra pixels between characters, and it matters more than people expect: a value of 1 or 2 makes most fonts noticeably easier to read on screen.

`DrawTextPro` adds an origin and a rotation, for damage numbers that float and spin, or text that follows a curve.

19.3 Measuring, which is how you lay anything out

There is no alignment parameter anywhere in raylib. You measure and subtract:

```
int w = MeasureText(txt, 20);           // default font
Vector2 size = MeasureTextEx(font, txt, 20, 2); // any font, gives w AND h

DrawText(txt, GetScreenWidth()/2 - w/2, y, 20, BLACK); // centred
DrawText(txt, GetScreenWidth() - w - 20, y, 20, BLACK); // right aligned
```

MeasureTextEx returning a **Vector2** is what lets you draw a background box that fits its text, which is most of what a UI is:

```
Vector2 s = MeasureTextEx(font, label, 20, 2);
DrawRectangle(x - 8, y - 6, (int)s.x + 16, (int)s.y + 12, Fade(BLACK, 0.7f));
DrawTextEx(font, label, (Vector2){ x, y }, 20, 2, RAYWHITE);
```

19.4 The string helpers

rtext also contains a small string library, which exists because doing this in plain C is tedious:

TextFormat("%i/%i", a, b)	sprintf, managed buffer
TextLength(s)	like strlen
TextSubtext(s, pos, len)	a slice
TextReplace(s, from, to)	search and replace
TextInsert(s, insert, pos)	
TextJoin(list, count, sep)	
TextSplit(s, delim, &count)	split into pieces
TextFindIndex(s, find)	position, or -1
TextToUpper / TextToLower	
TextToInteger / TextToFloat	parse
TextCopy(dst, src)	returns the byte count

WATCH OUT

TextFormat, TextSubtext, TextReplace and most of the others write into **internal rotating buffers**. The pointer is valid for drawing right now and becomes garbage after a few more calls. Never store one. If you need to keep the string, TextCopy it into your own array.

There is one exception that goes the other way: TextReplace in some versions returns heap memory you must free. When in doubt, check the comment in raylib.h — it says so per function.

19.5 Unicode, briefly

In UTF-8 one character is not one byte. TextLength counts *bytes*; the number of characters — codepoints — can be smaller:

```
int count = 0;
int *codepoints = LoadCodepoints(utf8Text, &count);
// count is the number of CHARACTERS, TextLength() is the number of BYTES
UnloadCodepoints(codepoints);
```

This matters the day you let a player type their name and then try to delete one character with backspace. Deleting one byte from a multi-byte character produces garbage. `GetCodepointPrevious()` gives you the correct amount to step back.

19.6 Where to get fonts

Any `.ttf` or `.otf` works. For games, pixel fonts hold up best at small sizes; Kenney's free asset packs and Google Fonts both have plenty that are free to ship. Load them with `LoadFontEx` at the exact size you will draw, and set `TEXTURE_FILTER_POINT` on `font.texture` if it is a pixel font, so it stays crisp.

RUN IT

Module 7 draws the same string through all three functions, shows measuring and centring live, runs through every string helper with its output, and counts bytes against codepoints on a UTF-8 string.

```
./cheatsheet/build.sh 7
```

The rest of rcore: saving, the window, other input

rcore is a third of the library. Chapters 2 to 5 covered the parts you need on day one; this is the rest, and it is mostly the plumbing a finished game needs.

20.1 Saving and loading

The simplest save file that works is text:

```
// writing
const char *text = TextFormat("score=%i\nlevel=%i\n", score, level);
SaveFileText("savegame.txt", (char *)text);

// reading
if (FileExists("savegame.txt"))
{
    char *text = LoadFileText("savegame.txt"); // heap allocated
    // ... parse it ...
    UnloadFileText(text);                      // you must free it
}
```

For binary data — a struct, a level, a screenshot — the pair is `SaveFileData` / `LoadFileData`, and the loader hands you the size:

```
int size = 0;
unsigned char *data = LoadFileData("level.bin", &size);
UnloadFileData(data);
```

WATCH OUT

Every `Load...` in this family allocates. Every one has a matching `Unload...`. This is the same discipline as textures, and it is easier to forget here because nothing appears on screen to remind you.

20.2 Paths, and the one that actually matters

<code>GetWorkingDirectory()</code>	where the program was <i>launched</i> from
<code>GetApplicationDirectory()</code>	where the executable <i>sits</i>
<code>ChangeDirectory(path)</code>	move the working directory
<code>GetFileName(path)</code>	"boss.png"
<code>GetFileNameWithoutExt(path)</code>	"boss"
<code>GetFileExtension(path)</code>	".png"
<code>GetDirectoryPath(path)</code>	the folder part
<code>IsFileExtension(path, ".png")</code>	
<code>FileExists / DirectoryExists</code>	
<code>GetFileLength(path)</code>	size in bytes
<code>LoadDirectoryFiles(path)</code>	list a folder

Most of those only parse the string and never touch the disk, so they are safe to call freely.

The distinction between the two directory functions is the source of the classic “it works in my IDE but not when I double-click it” bug. Relative paths resolve against the *working* directory, which is not necessarily where your game lives. The fix is one line at startup:

```
ChangeDirectory(GetApplicationDirectory());
```

or this project’s helper, `SearchAndSetResourceDir("resources")`.

20.3 Dropped files

```
if (IsFileDropped())
{
    FilePathList dropped = LoadDroppedFiles();

    for (unsigned int i = 0; i < dropped.count; i++)
        TraceLog(LOG_INFO, "dropped: %s", dropped.paths[i]);

    UnloadDroppedFiles(dropped);    // always, or you leak
}
```

Twenty lines and your level editor can open files by drag and drop. It is the cheapest tooling win in the library.

20.4 Random numbers

```
SetRandomSeed(1234);                // makes a run repeatable
int roll = GetRandomValue(1, 6);     // inclusive at both ends

int *deck = LoadRandomSequence(52, 1, 52); // shuffled, NO repeats
UnloadRandomSequence(deck);
```

`LoadRandomSequence` is the one people reimplement badly. Any time you need “pick these in a random order without repeating” — cards, spawn points, a shuffled playlist — it is already there.

Seeding deliberately is worth the habit: a fixed seed makes a bug reproducible, and it is how “daily challenge” levels work.

20.5 Logging

```
TraceLog(LOG_INFO, "level %i loaded", level);
TraceLog(LOG_WARNING, "texture missing, using placeholder");
TraceLog(LOG_ERROR, "save failed");

SetTraceLogLevel(LOG_WARNING); // quieten raylib's own chatter
```

raylib logs a lot at LOG_INFO, which is genuinely useful while learning — it is how you confirm a shader compiled or a texture loaded. Turn it down to LOG_WARNING for a release.

20.6 Managing the window at run time

ToggleFullscreen()	a real mode switch; can be slow
ToggleBorderlessWindowed()	fake fullscreen; instant, usually preferred
MaximizeWindow() / RestoreWindow()	needs FLAG_WINDOW_RESIZABLE
SetWindowSize(w, h) / SetWindowPosition(x, y)	
SetWindowTitle(text)	
SetWindowIcon(image)	
IsWindowResized()	true for the one frame it changed
IsWindowFocused()	pause the game when it goes false

And the monitor functions, for an options menu that offers real resolutions:

```
int m = GetCurrentMonitor();
GetMonitorName(m); GetMonitorWidth(m); GetMonitorHeight(m); GetMonitorRefreshRate(m);
```

WATCH OUT

GetScreenWidth() is the window in logical units; GetRenderWidth() is the actual framebuffer in pixels. On a HighDPI display they differ by a factor of two, and mixing them puts your HUD in the wrong place on exactly the machines you do not own.

20.7 Gamepads

```
if (IsGamepadAvailable(0))
{
    float x = GetGamepadAxisMovement(0, GAMEPAD_AXIS_LEFT_X); // -1 .. +1
    if (IsGamepadButtonPressed(0, GAMEPAD_BUTTON_RIGHT_FACE_DOWN)) Jump();
}
```

The button names are described by position, not by letter, because A on an Xbox pad is B on a Nintendo one: RIGHT_FACE_DOWN is the bottom button of the right cluster whatever the pad calls it.

WATCH OUT

Sticks never read exactly zero when released. Always apply a dead zone:

```
if (fabsf(x) < 0.15f) x = 0.0f;
```

Without it your character drifts slowly across the room while nobody is touching anything.

20.8 Touch and gestures

```
int count = GetTouchPointCount();
Vector2 p = GetTouchPosition(0);

int g = GetGestureDetected();    // GESTURE_TAP, GESTURE_SWIPE_LEFT, GESTURE_PINCH_OUT...
```

On a desktop the mouse counts as one touch point, so tap, hold, drag and swipe all work without a touchscreen — which means you can develop and test the mobile controls of a game on your laptop.

20.9 Clipboard and compression

```
SetClipboardText("share this");
const char *text = GetClipboardText();

unsigned char *packed = CompressData(raw, rawSize, &packedSize);
unsigned char *back    = DecompressData(packed, packedSize, &rawSize);
char *b64 = EncodeDataBase64(raw, rawSize, &outSize);
```

Compression plus base64 is how you make a save file or a level that a player can copy and paste to a friend as a block of text.

WATCH OUT

These return raylib-allocated memory. Free it with `MemFree()`, not `free()` — raylib can be built with a custom allocator, and mixing them crashes only on some builds.

RUN IT

Module 4 writes and reads a real save file, lists the path helpers with their output, shuffles a deck, compresses a buffer and copies to your clipboard. Module 1 covers the window and monitor side, and module 2 the input devices.

```
./cheatsheet/build.sh 4 1 2
```

raymath

raymath ships with raylib but is not included by `raylib.h`. You ask for it separately, and it has its own cheatsheet:

```
#include "raymath.h"
```

It is header-only and every function is small and inlined, so using it costs nothing at run time. Chapter 11 listed the handful you meet first; this is the whole shape of it.

21.1 The three ideas worth understanding

Everything else is arithmetic. These three are concepts.

21.1.1 Normalize: direction without length

```
Vector3 dir = Vector3Normalize(Vector3Subtract(target, me));
```

A normalized vector has length exactly 1, so it carries a *direction* and nothing else. Multiply it by a speed and you get a velocity; by a distance and you get an offset. Almost every movement line you will ever write is:

```
position = Vector3Add(position, Vector3Scale(dir, speed*dt));
```

WATCH OUT

Normalizing a zero vector divides by zero. raylib's version guards against it and returns zero, but the habit of checking `Vector3Length(v) > 0` before normalizing will save you elsewhere — and it is why the movement code in chapter 12 tests the length first.

21.1.2 Dot product: how much two directions agree

For two *normalized* vectors, the dot product is a single number:

+1	pointing the same way
0	perpendicular
-1	pointing opposite ways

That makes it the answer to a surprising number of game questions:

```
// Is the enemy in front of me, or behind?
Vector3 toEnemy = Vector3Normalize(Vector3Subtract(enemy, camera.position));
float facing = Vector3DotProduct(forward, toEnemy);
if (facing > 0.0f) { /* somewhere in front */ }

// Is it inside my 60-degree cone of vision?
if (facing > cosf(30.0f*DEG2RAD)) { /* it can see me */ }
```

That second one is a whole enemy vision system in one line, and it is far cheaper than computing angles.

21.1.3 Cross product: perpendicular to both

3D only. Give it two directions, get a third at right angles to both:

```
Vector3 right = Vector3CrossProduct(forward, (Vector3){ 0, 1, 0 });
```

That is the line from chapter 12 that gives you strafing. It is also how you compute a surface normal from a triangle's edges, which is how lighting knows which way a face points.

WATCH OUT

The cross product is not commutative: `cross(a, b)` points the opposite way to `cross(b, a)`. If your strafe keys are swapped, this is why, and swapping the arguments is the fix.

21.2 The families

21.2.1 Utils

<code>Lerp(a, b, t)</code>	blend: $t = 0$ gives a, $t = 1$ gives b
<code>Clamp(value, min, max)</code>	keep it in range
<code>Normalize(value, start, end)</code>	map a range onto 0..1
<code>Remap(v, inA, inB, outA, outB)</code>	map one range onto another
<code>Wrap(value, min, max)</code>	wrap around, for angles
<code>FloatEquals(a, b)</code>	float comparison that is not a lie

`Remap` is the quiet hero. Converting a health value to a bar width, a mouse position to a world coordinate, a distance to a volume — all one call:

```
float volume = Remap(distance, 0.0f, 20.0f, 1.0f, 0.0f); // near = loud
```

21.2.2 Vector2, Vector3, Vector4

The same set exists for each, with the obvious names: `Add`, `Subtract`, `Scale`, `Multiply`, `Divide`, `Length`, `LengthSqr`, `Distance`, `Normalize`, `DotProduct`, `Lerp`, `Negate`, `Min`, `Max`.

Plus a few that only make sense in one dimension:

- 2D only — `Vector2Angle`, `Vector2Rotate`
- 3D only — `Vector3CrossProduct`, `Vector3RotateByQuaternion`

THE KEY IDEA

Use `Vector3DistanceSqr` instead of `Vector3Distance` when you only want to compare two distances. Skipping the square root is free, and comparing squared distances gives the same answer. With hundreds of enemies it is a measurable win.

21.2.3 Matrix

A **Matrix** is a 4×4 grid of floats that encodes a transform: a translation, a rotation, a scale, or all three at once.

```
Matrix t = MatrixTranslate(x, y, z);
Matrix r = MatrixRotateY(angle);
Matrix s = MatrixScale(2, 2, 2);

Matrix combined = MatrixMultiply(MatrixMultiply(s, r), t); // scale, then rotate, then
move
model.transform = combined;
```

WATCH OUT

Matrix multiplication is not commutative. Rotating then translating puts the object somewhere completely different from translating then rotating. In raylib's convention the transforms apply left to right, so read `MatrixMultiply(s, r)` as “scale, then rotate”.

You rarely build these by hand. Where they turn up is `model.transform`, which lets you position a model without touching the draw call, and `DrawMeshInstanced`, which takes an array of them.

21.2.4 Quaternion

A quaternion is four floats that represent a rotation. They exist because Euler angles — yaw, pitch, roll — have two problems: gimbal lock (chapter 12), and the fact that blending between two of them takes a scenic route.

```
Quaternion q = QuaternionFromAxisAngle((Vector3){ 0, 1, 0 }, angle);
Vector3 spun = Vector3RotateByQuaternion(point, q);

Quaternion blended = QuaternionSlerp(from, to, t); // smooth, shortest path
```

`QuaternionSlerp` is the reason they exist: it interpolates between two orientations along the shortest arc, at constant speed. That is what every animation system and every smooth camera does internally.

For a first person camera, two angles are simpler and perfectly adequate — keep using yaw and pitch. Reach for quaternions when something needs to rotate freely in all three axes, like a spaceship or a physics object.

21.3 You do not have to use it

`raymath` is a convenience, not a requirement. `position.x += speed*dt` is a perfectly good line of code, and for simple 2D work writing the arithmetic out is often clearer than nesting three function calls.

Where it earns its place is 3D, where the vector operations get long and the chance of a typo in the third component is high.

RUN IT

Module 10 has two panels. In 2D, drag two vectors and watch add, subtract, dot, angle, lerp and rotate update live. Press TAB for the 3D panel, where the cross product and a quaternion rotation are drawn in space.

```
./cheatsheet/build.sh 10
```

Where to go from here

22.1 What you have now

Part I took you from an empty `main()` to a first person shooter. Part II walked the rest of the library module by module. Between the two, the code in this repository calls about 240 of raylib's 619 functions — and the rest is mostly depth inside modules you have already met, not new territory.

More to the point: you can now read `raylib.h` and understand what you are looking at. That was always the real goal, because that header is the documentation and nothing else is.

22.2 The examples are the next teacher

The official examples live in `build/external/raylib-master/examples`, 225 programs sorted by module. With this manual behind you they are readable rather than intimidating.

Worth opening first, in this order:

<code>core/core_3d_camera_first_person.c</code>	the official version of chapter 12
<code>models/models_first_person_maze.c</code>	the official version of chapter 14
<code>shapes/shapes_collision_area.c</code>	collision resolution, done cleanly
<code>textures/textures_sprite_anim.c</code>	sprite sheets in practice
<code>audio/audio_sound_multi.c</code>	overlapping sounds without clipping
<code>shaders/shaders_basic_lighting.c</code>	where the flat look finally goes away
<code>others/rlgl_standalone.c</code>	the layer under raylib, when you need it

22.3 Three things this manual did not cover

1. **raygui.** An immediate-mode GUI library in a single header, and it is *already in your examples folder* (`examples/*/raygui.h`). Buttons, sliders, checkboxes, dropdowns and text boxes, in the same style as the rest of raylib:

```
#define RAYGUI_IMPLEMENTATION
#include "raygui.h"

if (GuiButton((Rectangle){ 20, 20, 120, 30 }, "Start")) StartGame();
GuiSlider((Rectangle){ 20, 60, 200, 20 }, "vol", NULL, &volume, 0.0f, 1.0f);
```

For a pause menu or an options screen it turns a day of work into an hour.

2. **rlgl.** The layer raylib itself is built on: a thin abstraction over OpenGL that lets you push vertices directly. You want it the day you need a custom blend mode, instanced rendering or geometry raylib has no function for. It is `src/rlgl.h`, and it is standalone.

3. **Shipping.** Building for the web with Emscripten, packaging for Windows from Linux, bundling assets. The project wiki on raylib.com is the reference, and the examples folder ships a working `Makefile.Web` to start from.

22.4 The tools on raylib.com

Ramon Santamaria, raylib’s author, also writes small tools that pair with it. They are not part of the library and you download them separately:

rFXGen	generates retro sound effects with sliders
rTexPacker	packs sprites into an atlas
rTexViewer	views and converts textures
rGuiStyler	themes for raygui
rGuiLayout	lays out raygui screens and exports the C code
rIconPacker	builds .ico files

rFXGen in particular is worth ten minutes: chapter 15 showed you how those sounds are synthesised, and the tool lets you find one by ear instead of by arithmetic.

22.5 Habits worth keeping

- Update and draw stay separate. Update changes variables, draw paints them.
- Every speed is per second and gets multiplied by `dt`.
- Every `Load...` has an `Unload...`, outside the loop, on both sides.
- Detect collisions with one call; resolve them one axis at a time.
- Anything inside a `Begin/End` pair is in that pair’s space. The HUD goes outside.
- When something is invisible, check the draw order and the camera before suspecting anything clever.
- When a collision misbehaves, draw the box.
- Game state in a struct, plus a reset function.
- When stuck, open `raylib.h` and search. The answer is one `grep` away.

22.6 A last word

The gap between this manual and a game you would show someone is not more knowledge — it is repetitions. You have the loop, the cameras, the collisions, the maths, the sound and the shaders. What is left is choosing something small, finishing it, and discovering the twenty tiny problems that only show up when you try.

Make the mini Doom yours. Add a door that opens. Add a key that opens it. Add a sound when it does. Add a second level. Each of those is an afternoon, and each one teaches more than a chapter.

Cheat sheet

A.1 The skeleton

```
#include "raylib.h"
#include "raymath.h"           // only if you need vector maths

int main(void)
{
    SetConfigFlags(FLAG_VSYNC_HINT | FLAG_MSAA_4X_HINT);
    InitWindow(800, 450, "title");
    SetTargetFPS(60);

    // load textures, sounds, models here

    while (!WindowShouldClose())
    {
        float dt = GetFrameTime();

        // update

        BeginDrawing();
        ClearBackground(RAYWHITE);
        BeginMode3D(camera);
            // world
        EndMode3D();
        // HUD
        EndDrawing();
    }

    // unload everything here
    CloseWindow();
    return 0;
}
```

A.2 Window and timing

InitWindow(w, h, title)	creates window + OpenGL context
CloseWindow()	destroys it
WindowShouldClose()	true on ESC or the close button
SetConfigFlags(flags)	before InitWindow
SetTargetFPS(60)	cap the loop
GetFrameTime()	delta time, in seconds
GetTime()	seconds since InitWindow
GetFPS()	current frame rate
GetScreenWidth() / Height()	current window size

A.3 Drawing state

<code>BeginDrawing() / EndDrawing()</code>	wraps the whole frame
<code>ClearBackground(colour)</code>	wipe last frame
<code>BeginMode2D(cam2d) / EndMode2D()</code>	2D world coordinates
<code>BeginMode3D(cam3d) / EndMode3D()</code>	3D world coordinates + depth
<code>BeginTextureMode(rt) / EndTextureMode()</code>	draw into an offscreen image

A.4 2D drawing

<code>DrawRectangle(x, y, w, h, c)</code>	x,y is the top-left corner
<code>DrawRectangleRec(rec, c)</code>	takes a Rectangle
<code>DrawRectangleLinesEx(rec, thick, c)</code>	outline
<code>DrawCircle(cx, cy, r, c)</code>	cx,cy is the centre
<code>DrawLine(x1, y1, x2, y2, c)</code>	thin line
<code>DrawLineEx(v1, v2, thick, c)</code>	thick line
<code>DrawTriangle(v1, v2, v3, c)</code>	counter-clockwise!
<code>DrawText(txt, x, y, size, c)</code>	default font
<code>MeasureText(txt, size)</code>	width in pixels, for centring
<code>TextFormat("%i", n)</code>	sprintf that manages its buffer
<code>DrawFPS(x, y)</code>	debug frame rate

A.5 3D drawing

<code>DrawCube(centre, w, h, l, c)</code>	sizes are X, Y, Z
<code>DrawCubeV(centre, size, c)</code>	size as a Vector3
<code>DrawCubeWires(centre, w, h, l, c)</code>	edges only
<code>DrawSphere(centre, r, c)</code>	
<code>DrawCylinder(bottom, rTop, rBot, h, sides, c)</code>	rTop = 0 gives a cone
<code>DrawPlane(centre, size2, c)</code>	flat on XZ
<code>DrawLine3D(a, b, c)</code>	
<code>DrawGrid(slices, spacing)</code>	XZ floor grid at Y = 0
<code>DrawBillboard(cam, tex, pos, size, c)</code>	always faces the camera
<code>DrawModel(model, pos, scale, tint)</code>	loaded meshes
<code>DrawBoundingBox(box, c)</code>	collision debugging

A.6 Input

<code>IsKeyDown(KEY_W)</code>	every frame while held
<code>IsKeyPressed(KEY_SPACE)</code>	one frame, on the way down
<code>IsKeyReleased(KEY_SPACE)</code>	one frame, on the way up
<code>IsMouseButtonPressed(MOUSE_BUTTON_LEFT)</code>	
<code>GetMousePosition()</code>	screen pixels
<code>GetMouseDelta()</code>	movement this frame; for FPS cameras
<code>GetMouseWheelMove()</code>	
<code>DisableCursor() / EnableCursor()</code>	capture and release the pointer
<code>SetExitKey(KEY_NULL)</code>	stop ESC from closing the window

A.7 Cameras

```

Camera2D cam2 = { 0 };
cam2.target = player;                                     // world point
cam2.offset = (Vector2){ w/2.0f, h/2.0f };               // where it lands on screen
cam2.zoom    = 1.0f;                                     // NEVER leave this at 0

Camera3D cam3 = { 0 };
cam3.position = (Vector3){ 0, 1.7f, 6 };
cam3.target   = (Vector3){ 0, 1.7f, 0 };
cam3.up       = (Vector3){ 0, 1, 0 };
cam3.fovy     = 70.0f;
cam3.projection = CAMERA_PERSPECTIVE;

```

UpdateCamera(&cam, CAMERA_FREE)	built-in fly-around
UpdateCamera(&cam, CAMERA_ORBITAL)	spins around the target
UpdateCamera(&cam, CAMERA_FIRST_PERSON)	needs DisableCursor()
GetWorldToScreen(point3d, cam)	3D point to screen pixels
GetScreenToWorld2D(mouse, cam2)	mouse to 2D world
GetScreenToWorldRay(mouse, cam3)	mouse to a 3D ray, for picking

A.8 Collisions

CheckCollisionRecs(a, b)	2D box vs box
CheckCollisionCircles(ca, ra, cb, rb)	2D circle vs circle
CheckCollisionPointRec(point, rec)	mouse over a button
CheckCollisionBoxes(a, b)	3D AABB vs AABB
CheckCollisionBoxSphere(box, centre, r)	
GetRayCollisionBox(ray, box)	shooting and picking

A.9 raymath, the useful half

Vector3Add(a, b)	$a + b$
Vector3Subtract(a, b)	the vector from b to a
Vector3Scale(v, f)	stretch
Vector3Length(v)	magnitude
Vector3Normalize(v)	direction only, length 1
Vector3Distance(a, b)	
Vector3CrossProduct(a, b)	perpendicular to both
Vector3Lerp(a, b, t)	blend between two points
DEG2RAD, RAD2DEG	angle conversion constants

A.10 Audio

<code>InitAudioDevice()</code> / <code>CloseAudioDevice()</code>	before anything else in raudio
<code>LoadSound(file)</code> / <code>PlaySound(snd)</code>	short clips
<code>SetSoundPitch(snd, 1.2f)</code>	1.0 unchanged; vary it per shot
<code>SetSoundPan(snd, 0.5f)</code>	0 left, 1 right
<code>LoadSoundAlias(snd)</code>	second playback state, shared samples
<code>LoadMusicStream(file)</code>	long tracks
<code>UpdateMusicStream(music)</code>	every frame, or it stops
<code>LoadSoundFromWave(wave)</code>	from samples you built
<code>ExportWave(wave, "out.wav")</code>	write it to disk
<code>LoadAudioStream(rate, bits, ch)</code>	samples you push yourself
<code>SetMasterVolume(0.6f)</code>	everything at once

A.11 Shaders

<code>LoadShader(vsFile, fsFile)</code>	NULL means "use raylib's default"
<code>LoadShaderFromMemory(vs, fs)</code>	source as strings
<code>GetShaderLocation(shader, "uTime")</code>	once; returns <code>-1</code> if not found
<code>SetShaderValue(sh, loc, &v, TYPE)</code>	every frame
<code>BeginShaderMode(sh)</code> / <code>EndShaderMode()</code>	
<code>UnloadShader(sh)</code>	

GLSL names raylib fills in for you: `texture0`, `colDiffuse`, `fragTexCoord`, `fragColor`, and you must write `finalColor`.

A.12 Meshes, models and materials

<code>GenMeshCube</code> / <code>Sphere</code> / <code>Plane</code> / <code>Cylinder</code>	generated geometry
<code>GenMeshHeightmap(image, size)</code>	terrain from a greyscale image
<code>GenMeshCubicmap(image, size)</code>	a maze from a pixel map
<code>LoadModelFromMesh(mesh)</code>	wrap it with a default material
<code>model.materials[0].maps[MATERIAL_MAP_DIFFUSE].texture</code>	put a texture on it
<code>DrawModel</code> / <code>DrawModelEx</code> / <code>DrawModelWires</code>	
<code>DrawMeshInstanced(mesh, mat, transforms, n)</code>	thousands in one call
<code>LoadModelAnimations(file, &count)</code>	
<code>UpdateModelAnimation(model, anim, frame)</code>	
<code>GetMeshBoundingBox(mesh)</code>	for collisions

A.13 Images and render textures

GenImageColor / GradientLinear / GradientRadial	
GenImageChecked / WhiteNoise / PerlinNoise	
ImageResize / Crop / Flip / Rotate	all take Image *
ImageColorTint / Brightness / Contrast	
ImageDrawRectangle / Circle / Text	draw into the image
LoadRenderTexture(w, h)	a texture you draw into
BeginTextureMode(rt) / EndTextureMode()	before BeginDrawing
SetTextureFilter(tex, TEXTURE_FILTER_POINT)	crisp pixel art
GenTextureMipmaps(&tex)	stops distant textures shimmering

Remember the negative source height when drawing a render texture.

A.14 Files, saving and random

SaveFileText / LoadFileText / UnloadFileText	text saves
SaveFileData / LoadFileData / UnloadFileData	binary saves
FileExists / DirectoryExists / GetFileLength	
GetFileName / GetFileExtension / GetDirectoryPath	string work only
GetApplicationDirectory() / ChangeDirectory()	fix relative paths
IsFileDropped() / LoadDroppedFiles()	drag and drop
SetRandomSeed / GetRandomValue(min, max)	inclusive
LoadRandomSequence(n, min, max)	shuffled, no repeats
TraceLog(LOG_INFO, "x = %f", x)	print to the terminal
MemAlloc / MemFree	raylib's allocator; use it for raylib data

A.15 Loading and unloading

Always outside the loop. Always in pairs.

LoadTexture(file)	UnloadTexture(tex)
LoadImage(file)	UnloadImage(img)
LoadTextureFromImage(img)	UnloadTexture(tex)
LoadFont(file)	UnloadFont(font)
LoadModel(file)	UnloadModel(model)
LoadSound(file)	UnloadSound(snd)

When something goes wrong

B.1 It will not compile or link

Message	What it means
raylib.h: No such file	the compiler cannot find the header. Your <code>-I</code> path is wrong.
undefined reference to <code>'InitWindow'</code>	the header was found but the library was not. Check the <code>.a</code> path, and that it comes <i>after</i> your <code>.c</code> on the command line.
undefined reference to <code>'Vector3Add'</code>	you forgot <code>#include "raymath.h"</code> , or <code>-lm</code> for the maths library.
undefined reference to <code>'glfwInit'</code>	the system libraries are missing from the link: <code>-lpthread -lm -ldl -lrt -lX11</code> .
expected expression before <code>'{'</code>	a compound literal with no cast. Write <code>(Vector3){ 1, 2, 3 }</code> , not <code>{ 1, 2, 3 }</code> .

B.2 The window is there but the screen is black

1. **Is there a `BeginDrawing()` and an `EndDrawing()`?** Nothing shows until `EndDrawing()`.
2. **Is `ClearBackground()` the same colour as everything you draw?** Black text on a black background is a surprisingly popular first bug.
3. **Camera2D with `zoom == 0`.** A zero zoom scales the world to nothing. If you wrote `Camera2D cam = { 0 }` you must set `cam.zoom = 1.0f`.
4. **Is the camera inside a wall, or looking the wrong way?** Print `camera.position` and `camera.target` and see.
5. **Is the object behind the camera?** In 3D, things behind you are clipped away entirely.

B.3 The thing I drew is not there

- **Draw order.** Something opaque drawn afterwards is covering it.
- **Inside or outside the `Begin/End` pair?** A HUD element accidentally drawn inside `BeginMode3D` is somewhere out in the world, about one metre from the origin, and you will never find it by looking.
- **Triangle winding.** `DrawTriangle` with clockwise vertices is invisible. Swap two of them.
- **A texture that did not load.** You get a silent no-op, not a crash. Check with `IsTextureValid()`, and remember that paths are relative to where you *launched* the program.

- **Alpha zero.** `BLANK` and a badly built `Color` both draw perfectly invisible pixels.

B.4 It moves wrong

- **Too fast or too slow on another machine** — a `GetFrameTime()` is missing somewhere.
- **Faster diagonally** — the movement vector is not normalized.
- **Flies into the sky when looking up** — you used the full `forward` to walk instead of the flattened one.
- **Camera snaps when looking straight up** — the pitch is not clamped. Gimbal lock.
- **Vertical movement inverted** — you are in screen space where `Y` grows down, or you mixed the sign on the mouse pitch.
- **One step per key tap** — `IsKeyPressed` where you wanted `IsKeyDown`.

B.5 The collisions are wrong

1. **Draw the box.** `DrawBoundingBox()` in 3D, `DrawRectangleLinesEx()` in 2D. Most collision bugs become obvious the instant you can see the box.
2. **Eyes versus body.** `camera.position` is head height. The collision box belongs around the body, half a body lower.
3. **Stale box.** If you build the box before moving and test it after, you are testing last frame's position.
4. **Getting stuck on corners** — you are testing both axes at once. Split them.
5. **Tunnelling through walls at speed** — the object moved further in one frame than the wall is thick. Clamp `dt`, or move in smaller steps.

B.6 It flickers

Two surfaces occupying exactly the same plane fight over the same pixels; the depth buffer cannot decide between them, and the result shimmers as the camera moves. It is called z-fighting. Move one of them a hair: `0.01f` is invisible to the eye and decisive to the GPU.

B.7 It runs at four frames per second

- **Loading inside the loop.** A `LoadTexture()` or `LoadModel()` in the frame loop will do this immediately.
- **Tens of thousands of draw calls.** Each `DrawCube` is cheap, but a 200×200 grid of them is 40,000 per frame. Only draw what is near the player.
- **`DrawPixel` in a loop.** It is a whole draw call per pixel. The header warns you about this in its own comment.
- **You are looking at a debug build.** `make config=release_x64` is a great deal faster.

B.8 When none of that helps

Print things. `DrawText(TextFormat(...))` on screen, or `TraceLog(LOG_INFO, "x = %f", x)` to the terminal. Graphics bugs look mysterious and are almost always a number being different from what you assumed. Put the number on the screen and the mystery usually lasts about ten seconds.

The lessons, at a glance

Every file compiles on its own and runs on its own. Build any of them with the folder's `build.sh` script and the lesson number.

C.1 course2d — the fundamentals

#	File	What it shows
1	<code>lesson01_window.c</code>	the frame loop, Begin/EndDrawing, immediate mode
2	<code>lesson02_shapes.c</code>	screen coordinates, every basic shape, alpha, draw order
3	<code>lesson03_input.c</code>	pressed vs down, and delta time against per-frame movement
4	<code>lesson04_collision2d.c</code>	CheckCollisionRecs and per-axis resolution
5	<code>lesson05_textures.c</code>	Image vs Texture, sprite sheets, timer-driven animation
6	<code>lesson06_camera2d.c</code>	Camera2D, world vs screen space, HUD outside the pair

C.2 course3d — the 3D half

#	File	What it shows
1	<code>lesson01_axes.c</code>	X, Y and Z, from four different viewpoints
2	<code>lesson02_camera.c</code>	the five camera fields, moved live with the keyboard
3	<code>lesson03_modes.c</code>	the four built-in UpdateCamera modes
4	<code>lesson04_movement.c</code>	yaw, pitch, forward and right, built by hand
5	<code>lesson05_collision.c</code>	AABB collision with the three axis overlaps shown
6	<code>lesson06_scene.c</code>	a closed room whose walls actually stop you
7	<code>lesson07_mini_doom.c</code>	the lot: text map, grid collision, shooting, HUD, minimap

C.3 cheatsheet — one example per module (Part II)

#	File	Module, and what it shows
1	m1_rcore_window.c	rcore: window state, flags, monitors, cursor, timing
2	m2_rcore_input.c	rcore: keyboard, mouse, gamepad, touch, gestures, file drop
3	m3_rcore_shaders.c	rcore: GLSL from memory, uniforms, BeginShaderMode
4	m4_rcore_misc.c	rcore: save files, paths, random, logging, clipboard, compression
5	m5_rshapes.c	rshapes: every 2D shape, splines, collision tests
6	m6_rtextures.c	rtextures: image generation and editing, render textures, colour
7	m7_rtext.c	rtext: fonts, measuring, string helpers, UTF-8
8	m8_rmodels.c	rmodels: meshes, models, materials, billboards, ray picking
9	m9_raudio.c	raudio: synthesised sounds, streamed music, live audio streams
10	m10_raymath.c	raymath: vectors, dot and cross, lerp, matrices, quaternions

None of them loads an external asset: every image, sound and tune is generated in code, so each file compiles and runs on its own.

C.4 Which chapter goes with which example

Chapter	Subject	Example
15	sound and music	./cheatsheet/build.sh 9
16	shaders	./cheatsheet/build.sh 3
17	meshes, models, materials	./cheatsheet/build.sh 8
18	images, render textures, colour	./cheatsheet/build.sh 6
19	fonts and text	./cheatsheet/build.sh 7
20	saving, window, other input	./cheatsheet/build.sh 4, 1, 2
21	raymath	./cheatsheet/build.sh 10

C.5 Controls, by lesson

Lesson	Keys
2d/2	SPACE swaps the draw order
2d/3	WASD or arrows move, SPACE pulses, left click teleports
2d/4	WASD move, SPACE toggles solid and ghost walls
2d/5	SPACE pauses, UP/DOWN change the animation speed
2d/6	WASD move, wheel zooms, Q/E rotate, R resets
3d/1	1, 2, 3, 4 change viewpoint, SPACE toggles the helper cubes
3d/2	arrows and PgUp/PgDn move the eye, IJKL and U/O move the target, +/- fovy, P projection, R reset
3d/3	TAB cycles camera mode
3d/4	mouse looks, WASD moves, SHIFT runs, SPACE/CTRL fly, TAB frees the mouse
3d/5	WASD on X and Z, Q/E height, R reset, wheel zooms
3d/6	WASD move, mouse look, SHIFT run, TAB free mouse, R reset
3d/7	WASD move, mouse look, SHIFT run, LEFT CLICK shoot, R restart, TAB free mouse
cs/1	F fullscreen, B borderless, M maximize, C cursor, P screenshot
cs/2	type anything, drag a file in, plug in a gamepad
cs/3	0-3 pick a shader, mouse moves the light, SPACE pauses time
cs/4	R random, S save, L load, D delete, C clipboard, Z compress
cs/5	drag the blue box and the red points, TAB spline type
cs/6	1-6 generated image, T texture filter, SPACE render texture
cs/7	UP/DOWN size, LEFT/RIGHT spacing, R rotate, TAB text
cs/8	right drag orbit, wheel zoom, TAB mode, LEFT CLICK pick
cs/9	1 beep, 2 laser, 3 boom, M music, SPACE stream, UP/DOWN volume
cs/10	drag the two white dots, TAB switches the 2D/3D panel